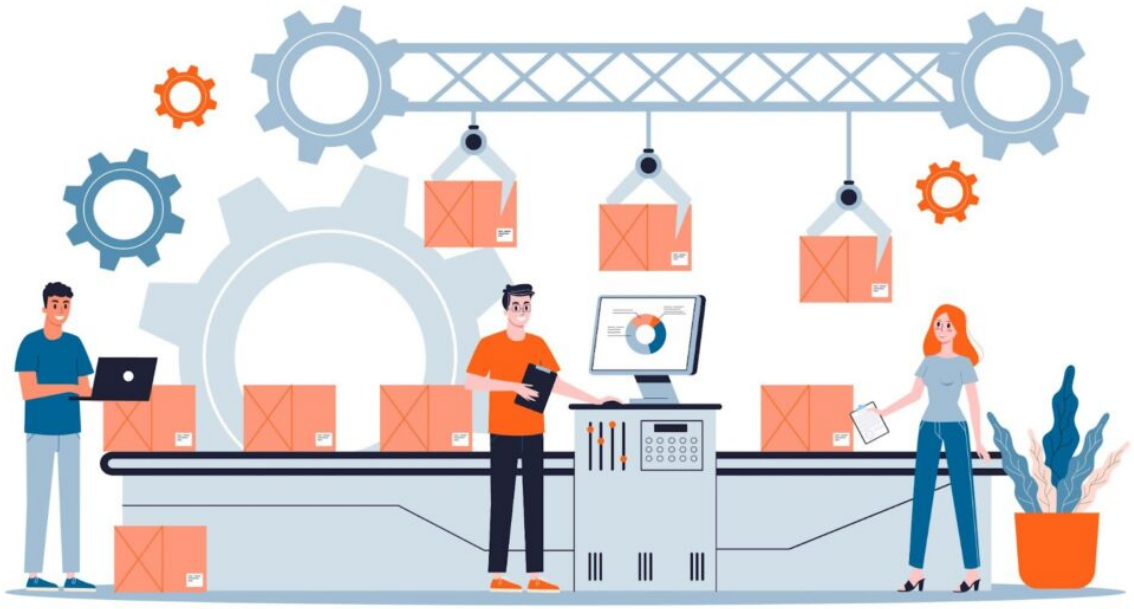
Rapport final - Génie Industriel

Optimisation du positionnement de machines pour minimiser le coût de production dans un atelier



Groupe 6 - Mohammed Chali, Valéria Ko, Blandine Piret, Clélie Thomas-Suhner

24 novembre 2025

Table des matières

1 Présentation du problème **4**

2 Modélisation du problème **4**
2.1 Paramètres et données du problème 4
2.2 Variable de décision 5
2.3 Fonction objectif et contraintes 6
2.4 Fonction de calcul de coût 6

3 Solutions initiales **7**
3.1 Méthode Gloutonne 7
3.1.1 Principe 7
3.1.2 Pseudo-code de la méthode gloutonne 7
3.1.3 Résultat obtenu 7
3.2 Méthode Gloutonne Aléatoire 8
3.2.1 Principe 8
3.2.2 Pseudo-code de la méthode gloutonne aléatoire 8
3.2.3 Résultat obtenu 8

4 Méthodes de recherche locale **9**
4.1 Méthode de descente locale : 2-swap 9
4.1.1 Principe 9
4.1.2 Pseudo-code de la méthode de descente locale 9
4.1.3 Résultats 9
4.2 Méthode d'insertion à forte pente 10
4.2.1 Principe 10
4.2.2 Pseudo-code de la méthode d'insertion à forte pente 11
4.2.3 Résultats 11

5 Méthodes métaheuristiques **12**
5.1 Méthode taboue 12
5.1.1 Présentation 12
5.1.2 Principe général de la méthode 12
5.1.3 Types de voisinage 13
5.1.4 Pseudo-code de la méthode taboue 13
5.1.5 Résultats et influence des paramètres 15
5.2 Méthode du recuit simulé 18
5.2.1 Présentation 18
5.2.2 Principe général de la méthode 18
5.2.3 Pseudo-code du recuit simulé 19
5.2.4 Choix des paramètres de la méthode 19

5.2.5 Résultats et analyse 20
5.2.6 Quelques remarques importantes 21
5.3 Algorithme génétique 21
5.3.1 Présentation 21
5.3.2 Principe général de la méthode 22
5.3.3 Pseudo-code de l'algorithme génétique 24
5.3.4 Résultats et influence des paramètres 27
5.3.5 Remarque importante 30
5.4 Algorithme de la colonie de fourmis (ACO) 30
5.4.1 Présentation 30
5.4.2 Principe général de la méthode 30
5.4.3 Pseudo-code de l'algorithme ACO 32
5.4.4 Paramètres ACO : Rôle et interprétation 33
5.4.5 Justification du choix des paramètres à tester 33
5.4.6 Résultats et influence des paramètres 34

6 Gestion de la diversité dans l'algorithme génétique 37
6.1 Distance de Hamming 37
6.2 Goodness Score (GS) 37
6.3 Impact sur la qualité de la solution 38
6.4 Impact sur le temps de calcul 38

7 Analyse et comparaison des résultats 39
7.1 Jeu de résultats obtenu pour la petite instance 39
7.2 Analyse des performances pour la petite instance 40
7.3 Jeu de résultats obtenu pour la grande instance 41
7.4 Analyse des performances pour la grande instance 44
7.5 Graphes de comparaison 45

8 Organisation du travail et Coordination 47

9 Conclusion 49

1 Présentation du problème

Il s'agit d'un **problème de réorganisation d'un atelier**. L'identification des besoins de l'entreprise nous amène à chercher une minimisation des flux logistiques. L'atelier dispose de N emplacements pour placer $N-1$ machines. L'emplacement 0 correspond à l'entrée (par laquelle doit commencer chaque production), et l'emplacement $N+1$ correspond à la sortie (par laquelle doit se terminer chaque production).

De plus :

— L'entreprise dispose d'un ordre de production pour l'année à venir, avec le nombre de ventes attendues pour chaque produit.

— Pour chaque produit, il est nécessaire de passer par certaines machines dans un ordre précis.

— On dispose de la matrice des distances entre tous les emplacements.

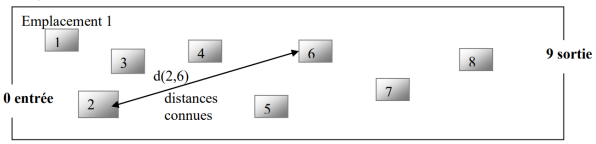


FIGURE 1 – Schéma représentant le problème sur une petite instance.

Notre modélisation du problème est la suivante :

— Les solutions sont représentées par une **liste des machines indexées par emplacement**.

— La **fonction de coût** est la distance totale parcourue pour produire l'ensemble de l'ordre de production, c'est-à-dire la somme des distances entre les emplacements pour chaque produit, pondérées par le volume des ventes.

— De plus, nous introduisons une machine fictive pour avoir le même nombre de machines que d'emplacements et ainsi simplifier le problème.

Objectif : minimiser cette fonction de coût afin d'optimiser les flux de l'atelier.

2 Modélisation du problème

2.1 Paramètres et données du problème

Comme paramètres du modèle, on a N emplacements sur lesquels on veut placer $N-1$ machines ($+1$ fictive) et M produits.

On a comme données la distance entre les emplacements, la sortie et l'entrée deux-à-deux. On note D la matrice des distances, avec : $(d_{i,j})_{(i,j)\in[0,n+1]} \in \mathbb{N}$

$$D = \begin{bmatrix} d_{0,0} & d_{0,1} & d_{0,2} & \cdots & d_{0,N+1} \\ d_{1,0} & d_{1,1} & d_{1,2} & \cdots & d_{1,N+1} \\ \vdots & & \ddots & & \vdots \\ d_{N+1,0} & d_{N+1,1} & d_{N+1,2} & \cdots & d_{N+1,N+1} \end{bmatrix}$$

Au delà de cette donnée, on a aussi la matrice Z décrit l'ordre de passage des produits par les machines, avec : $(z_{i,j})_{(i,j)\in[1,M]\times[1,N]} \in [0, N-1]$

$$Z = \begin{bmatrix} z_{1,1} & z_{1,2} & \cdots & z_{1,N} \\ z_{2,1} & z_{2,2} & \cdots & z_{2,N} \\ \vdots & & \ddots & \\ z_{M,1} & z_{M,2} & \cdots & z_{M,N} \end{bmatrix}$$

Cette matrice représente les données de l'ordre de fabrication des produits en fonction des machines qu'ils utilisent et à quelle étape. Machine utilisée à l'étape j dans la fabrication du produit i.

On dispose aussi de la donnée du nombre de ventes pour chaque produit. Qui donnera le poids de la distance parcourue pour la fabrication du produit, , avec : $(v_i)_{i\in[1,M]} \in \mathbb{N}$

$$V = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_M \end{bmatrix}$$

2.2 Variable de décision

Dans le cadre de notre problème, on cherche à minimiser la distance parcourue par les produits pour être fabriqués pondérée par le nombre de ventes des produits individuellement. En effet, plus un produit est fabriqué, plus il est intéressant de minimiser la distance qu'il parcourt. Pour ce faire, la variable sur laquelle on peut influer est la position des machines dans l'atelier, on aura donc comme variables des **variables binaires qui indiquent la présence ou non d'une machine sur un emplacement** :

$$x_{ij} = \begin{cases} 1 & \text{si la machine } i \text{ est placée à l'emplacement } j, \\ 0 & \text{sinon.} \end{cases}$$

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,N} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,N} \\ \vdots & & \ddots & \\ x_{N-1,1} & x_{N-1,2} & \cdots & x_{N-1,N} \end{bmatrix}$$

2.3 Fonction objectif et contraintes

Dans notre cas, nous avons choisi une fonction objectif qui est à minimiser et qui représente **la distance totale parcourue** pour assurer la production **pondérée par le nombre moyen de chacun des produits**. La fonction objective est alors :

$$\min(\sum_{k=1}^{M} v_k(\sum_{n=1}^{N-1}\sum_{j_1=1}^{N}\sum_{j_2=1}^{N} x_{z_{k,n},j_1}\, x_{z_{k,n+1},j_2}\, d_{j_1 j_2}))$$

Au delà de cette fonction objectif, la résolution de notre problème d'optimisation est aussi soumis à des contraintes, que sont :

— Il ne faut qu'une machine par emplacement au plus :

$$\sum_i x_{ij} \leq 1$$

— Il faut avoir exactement un emplacement pour chaque machine :

$$\sum_j x_{ij} = 1$$

— Contraintes d'incompatibilité :

$$x_{1,3} = 0, \quad x_{2,4} = 0$$

2.4 Fonction de calcul de coût

La fonction de calcul de coût correspond au **calcul de la fonction objectif précédemment détaillée**, associée à une affectation donnée des machines aux emplacements.

Son principe est simple : pour chaque produit, on reconstitue le chemin complet de fabrication, depuis l'entrée de l'atelier (emplacement 0) jusqu'à la sortie (emplacement 9), en suivant l'ordre des machines nécessaires à sa production.

La distance totale parcourue est obtenue en **sommant les distances successives entre les emplacements** correspondants, selon la matrice des distances. Cette distance est ensuite **pondérée par le volume de production** associé à chaque produit. La fonction renvoie alors la somme de ces coûts pondérés pour l'ensemble des produits, correspondant ainsi bien à la fonction objectif à minimiser.

Il est essentiel de **conserver la même fonction de coût pour l'ensemble des méthodes** (gloutonnes, heuristiques et métaheuristiques) afin d'assurer une comparaison cohérente des performances de chacune. Cela garantit que les différences observées d'efficacité entre les méthodes proviennent bien de leurs mécanismes de recherche, et non d'une modification du mode d'évaluation.

La complexité temporelle de la fonction de calcul du coût est $\mathcal{O}(N^2)$

3 Solutions initiales

3.1 Méthode Gloutonne

3.1.1 Principe

L'algorithme glouton est une méthode de construction de solution pour l'affectation des machines à des emplacements. Il fonctionne en parcourant les machines selon un ordre donné (**ici le produit de référence**) et en affectant à chaque machine l'emplacement qui minimise localement le coût, calculé comme le produit de la distance depuis la machine précédente et du volume associé. Les emplacements déjà utilisés ou incompatibles sont exclus à chaque étape. Une fois toutes les machines prioritaires affectées, les emplacements restants sont attribués aux machines non encore placées de manière arbitraire.

Cet algorithme permet d'obtenir rapidement une solution initiale de qualité raisonnable, qui peut ensuite servir de **point de départ pour des méthodes d'optimisation locales ou métaheuristiques**.

3.1.2 Pseudo-code de la méthode gloutonne

Algorithm 1 Algorithme de construction gloutonne

Input: Machines, emplacements, contraintes, matrice des distances D, produit de référence, volume associé

Output: Affectation des machines

affectation $\leftarrow \emptyset$; dispo $\leftarrow$ emplacements disponibles ; prev $\leftarrow 0$;
for *chaque machine m dans produit de référence* **do**
  candidats $\leftarrow$ $e \in dispo \mid e \notin contraintes[m]$; choix $\leftarrow \text{argmin}_{e \in candidats}\big(D[prev][e] \times volume\big)$; affectation[m] $\leftarrow$ choix ; Retirer *choix* de *dispo* ; prev $\leftarrow$ choix ;
end
for *chaque machine m non encore affectée* **do**
  affectation[m] $\leftarrow$ élément restant dans *dispo* ;
end
return affectation ;

Si m est le nombre de machines et e le nombre d'emplacements, la complexité est de l'ordre de $\mathcal{O}(m \times e)$, car pour chaque machine on évalue tous les emplacements encore disponibles.

3.1.3 Résultat obtenu

```
=== Solution Gloutonne ===
{'M2': 2, 'M1': 8, 'M6': 7, 'M3': 1, 'M7': 5, 'M5': 6, 'M4': 4, 'M8': 3}
Coût = 1772607
```

3.2 Méthode Gloutonne Aléatoire

3.2.1 Principe

La méthode gloutonne aléatoire est une **variante de la méthode gloutonne classique**. Conçue sur le même principe d'une construction progressive de la solution par choix localement optimaux, elle introduit une notion d'aléa contrôlé afin de diversifier la recherche et d'éviter de tomber systématiquement sur la même solution. Cela permet de créer une diversité dans les points de départ de recherche des méthodes heuristiques comme métaheuristiques. Ce principe, appelé **parachutage**, favorise l'exploration de voisinages variés de solutions, augmentant ainsi la probabilité d'atteindre une solution plus intéressante.

Le principe diffère de la méthode gloutonne usuelle : à chaque étape, on ne choisit plus le meilleur candidat unique, mais on forme une *liste restreinte de bons candidats* (*Restricted Candidate List*, ou RCL). Cette liste regroupe les k meilleures options locales selon la solution courante, et l'une d'entre elles est choisie aléatoirement. La procédure est répétée jusqu'à ce que toutes les machines soient affectées. Ainsi, cette méthode combine les avantages du glouton (qualité) et de l'aléatoire (diversité).

3.2.2 Pseudo-code de la méthode gloutonne aléatoire

Algorithm 2 Algorithme de construction gloutonne aléatoire

Input: Machines, emplacements, contraintes, matrice des distances D, produit de référence, volume associé, taille de la RCL rcl_size

Output: Affectation des machines

affectation $\leftarrow \emptyset$ dispo $\leftarrow$ emplacements disponibles prev $\leftarrow 0$
for *chaque machine* m **do**
  candidats $\leftarrow \{e \in dispo \mid e \notin contraintes[m]\}$ Trier candidats par coût croissant : $cot(e) = D[prev][e] \times volume$ RCL $\leftarrow$ premiers $\min(rcl_size, |candidats|)$ éléments triés choix $\leftarrow$ élément choisi aléatoirement dans RCL affectation$[m] \leftarrow$ choix Retirer *choix* de *dispo* prev $\leftarrow$ choix
end
for *chaque machine non encore affectée* **do**
  affectation$[m] \leftarrow$ emplacement restant dans *dispo*
end
return affectation

Il faut trier les candidats pour chaque machine, ce qui coûte $\mathcal{O}(e \log(e))$, puis sélectionner un élément dans la RCL. La complexité totale est donc de l'ordre de $\mathcal{O}(m \times e \log(e))$.

3.2.3 Résultat obtenu

```
=== Solution Gloutonne Aléatoire ===
{'M1': 1, 'M2': 5, 'M3': 7, 'M4': 9, 'M5': 4, 'M6': 6, 'M7': 3, 'M8': 2}
Coût = 1 716 999
```

4 Méthodes de recherche locale

4.1 Méthode de descente locale : 2-swap

4.1.1 Principe

La **recherche locale par descente** est une méthode d'optimisation itérative qui part d'une solution initiale et explore son voisinage afin d'améliorer progressivement le coût. Le principe général est simple : à chaque itération, on examine toutes les solutions voisines obtenues par de petites modifications locales.

Le 2-Swap consiste à échanger deux affectations de machines entre elles. Si un échange permet de **réduire le coût total**, il est accepté, et la nouvelle solution devient la solution courante. Ce processus est répété jusqu'à ce qu'aucun échange ne puisse plus améliorer la solution, garantissant ainsi l'obtention d'**un minimum local**. L'algorithme 2-Swap explore donc toutes les paires possibles de machines, vérifie la validité des échanges selon les contraintes, évalue le coût de la nouvelle solution, et conserve la meilleure configuration trouvée.

4.1.2 Pseudo-code de la méthode de descente locale

Algorithm 3 Algorithme de recherche locale par descente (2-Swap)

Entrée: Solution initiale S_0, contraintes, données de distances et de volumes
Sortie : Meilleure solution S_{opt} et son coût $COUT_{opt}$

$S_{opt} \leftarrow S_0$ $COUT_{opt} \leftarrow calculer_cout(S_0)$ $amelioration \leftarrow vrai$

while *amelioration* **do**
  $amelioration \leftarrow faux$
  for *chaque paire* (i, j) *de machines* **do**
    if *échange* (i, j) *valide* **then**
      $S_{temp} \leftarrow$ copie de S_{opt} échanger les emplacements de i et j dans S_{temp}
      $COUT_{temp} \leftarrow calculer_cout(S_{temp})$
      if $COUT_{temp} < COUT_{opt}$ **then**
        $S_{opt} \leftarrow S_{temp}$ $COUT_{opt} \leftarrow COUT_{temp}$ $amelioration \leftarrow vrai$ **break**
      end
    end
  end
end
return $S_{opt}, COUT_{opt}$

Pour chacun des $O(N^2)$ échanges, il faut évaluer le coût de la solution modifiée, dont la complexité est $O(N^2)$. Ainsi, **la complexité totale est de** $O(N^4)$.

4.1.3 Résultats

On observe dans le tableau ci-dessous que **la solution gloutonne classique est déjà proche d'un optimum local**, avec un gain total très faible de seulement 0,24 % après

l'application du 2-swap. Cela indique que l'heuristique gloutonne construit directement une solution de bonne qualité, laissant peu de marge d'amélioration pour la recherche locale.

À l'inverse, la solution gloutonne aléatoire part d'un coût initial plus élevé, mais bénéficie d'une amélioration beaucoup plus significative grâce à la descente locale, avec un gain de 10,96 %. Cela montre que l'**introduction d'aléa permet d'explorer des régions différentes de l'espace de solutions**, plus éloignées des optima locaux.

Solution	**Coût initial**	**Coût final**	**Gain total (%)**	**Temps (ms)**
Gloutonne	1 716 999	1 712 860	0.24	1.18
Gloutonne aléatoire	1 782 218	1 586 943	10.96	2.26

TABLE 1 – Résultats du 2-swap sur les solutions gloutonne et gloutonne aléatoire

4.2 Méthode d'insertion à forte pente

4.2.1 Principe

La **méthode d'insertion à forte pente** est une approche de recherche locale qui vise à améliorer une solution existante en explorant des déplacements unitaires de machines. Contrairement au 2-Swap, où deux machines échangent leurs positions, lors d'une insertion, on retire une machine de son emplacement actuel pour la réinsérer ailleurs tout en respectant les contraintes. L'idée de la **forte pente** vient du fait que, parmi tous les déplacements possibles testés à chaque itération, l'algorithme **sélectionne celui qui permet d'avoir la plus grande amélioration de coût**. Une fois ce déplacement appliqué, un nouveau balayage complet est effectué pour chercher une meilleure insertion. Ce processus se poursuit jusqu'à ce qu'aucun déplacement ne permette de réduire davantage le coût, garantissant ainsi la convergence vers un **minimum local**.

Cette méthode est plus coûteuse que le 2-Swap, car elle examine un plus grand nombre de configurations potentielles, mais elle est souvent plus efficace pour affiner une solution et atteindre un meilleur optimum local.

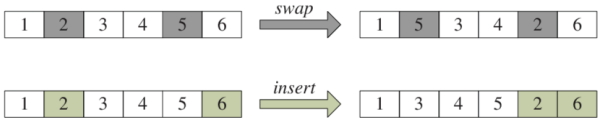


FIGURE 2 – Illustration des mouvements d'insertion et de 2-swap.

4.2.2 Pseudo-code de la méthode d'insertion à forte pente

Algorithm 4 Algorithme de recherche locale par insertion à forte pente

Entrée: Solution initiale S_0, contraintes, données de distances et de volumes
Sortie : Meilleure solution S_{opt} et son coût $COUT_{opt}$

$S_{opt} \leftarrow S_0$ $COUT_{opt} \leftarrow calculer_cout(S_0)$
while *amélioration possible* **do**
  $meilleur_gain \leftarrow 0$ $meilleure_insertion \leftarrow None$
  for *chaque machine* m **do**
    for *chaque emplacement* e *valide* **do**
      $S_{temp} \leftarrow$ copie de S_{opt} déplacer m vers e dans S_{temp} $COUT_{temp} \leftarrow calculer_cout(S_{temp})$
      if $COUT_{temp} < COUT_{opt}$ **then**
        $gain \leftarrow COUT_{opt} - COUT_{temp}$ **if** $gain > meilleur_gain$ **then**
          $meilleur_gain \leftarrow gain$ $meilleure_insertion \leftarrow (m, e, S_{temp})$
        end
      end
    end
  end
  if $meilleure_insertion \neq None$ **then**
    appliquer la meilleure insertion trouvée $S_{opt} \leftarrow S_{temp}$ $COUT_{opt} \leftarrow COUT_{temp}$
  end
  else
    break
  end
end
return $S_{opt}, COUT_{opt}$

À chaque itération, l'algorithme examine toutes les machines (N) et pour chacune, tous les emplacements valides (N). Pour chaque déplacement potentiel, on doit évaluer le coût de la solution obtenue, dont la complexité est $O(E) = O(N^2)$. Ainsi, **la complexité totale est de** $O(N^4)$.

4.2.3 Résultats

D'après le tableau 2, on constate tout d'abord que la méthode d'insertion est beaucoup plus efficace que le 2-swap pour améliorer les deux solutions. En partant d'une solution gloutonne classique, le coût passe de 1 716 999 à 1 608 464, soit un gain de 6,32 %, ce qui montre que le voisinage d'insertion explore des améliorations plus profondes que le voisinage par échange simple.

Pour la solution gloutonne aléatoire, l'amélioration est encore plus marquée : le coût final atteint 1 517 454, avec un gain total de 14,86 %.

Solution	Coût initial	Coût final	Gain total (%)	Temps (ms)
Gloutonne	1 716 999	1 608 464	6.32	6.01
Gloutonne aléatoire	1 782 218	1 517 454	14.86	10.94

TABLE 2 – Résultats sur les solutions gloutonne et gloutonne aléatoire

5 Méthodes métaheuristiques

Les méthodes de recherche locale présentées jusqu'ici relèvent des **méthodes heuristiques** qui sont certes efficaces mais restent locales et peuvent se **bloquer dans des optimums locaux**.

Afin de dépasser ces limites, les métaheuristiques permettent de mieux équilibrer la recherche entre **exploration** (diversification des solutions) et **exploitation** (qualité des solutions). Elles incluent souvent une recherche locale à l'intérieur mais en adoptant différentes stratégies pour éviter de se bloquer dans des optimum locaux, dans le but d'**atteindre des solutions proches de l'optimum global**.

5.1 Méthode taboue

5.1.1 Présentation

La **méthode taboue** est une métaheuristique qui repose sur de la recherche locale améliorée : à chaque itération, la méthode explore l'ensemble des solutions voisines d'une solution courante et se déplace vers la meilleure d'entre elles, même si cette dernière est moins bonne que la solution actuelle.

Pour **éviter les retours en arrière et le piégeage dans les minima locaux**, la méthode conserve une mémoire des mouvements récemment effectués, appelée liste taboue. Cette mémoire interdit temporairement certaines modifications, favorisant ainsi une exploration plus large de l'espace de recherche.

5.1.2 Principe général de la méthode

1. **Initialisation** : La méthode débute à partir d'une solution initiale, dans notre cas issue d'une méthode gloutonne (classique ou aléatoire). Cette solution permet d'amorcer la recherche dans une zone prometteuse de l'espace des solutions. Cette première évaluation permet de fixer un point de référence à partir duquel les améliorations seront mesurées.
2. **Voisinage** : À partir de la solution courante, on génère un ensemble de voisins en effectuant des échanges de positions entre deux machines. Chaque voisin représente une nouvelle configuration potentielle obtenue par un simple échange.
3. **Mémoire taboue** : Afin d'éviter **blocage** (dans des optimums locaux) ou **bouclage**, la méthode conserve une mémoire des **derniers mouvements effectués**,

appelée liste taboue. Chaque mouvement (échange de deux machines) reste interdit pendant un certain nombre d'itérations, défini par une certaine durée. Cette mémoire à court terme permet d'éviter de revisiter les mêmes solutions, de diversifier la recherche, et de maintenir une dynamique d'exploration.

4. **Condition d'arrêt** : La recherche s'interrompt lorsque le nombre maximal d'itérations (*NbMaxIter*) est atteint, ou lorsque le coût ne s'améliore plus pendant un certain nombre d'itérations consécutives (*NbMaxIterStable*). Ce double critère assure une convergence contrôlée tout en limitant le temps de calcul.
5. **Paramètres d'instanciation** : Lors de l'implémentation, plusieurs paramètres influencent directement la qualité et la rapidité de la recherche : Nombre maximal d'itérations, Nombre d'itérations sans amélioration avant arrêt, Taille de la liste taboue, Solution de départ, Fonction de coût.

5.1.3 Types de voisinage

Dans la méthode taboue, plusieurs structures de voisinage peuvent être utilisées pour explorer l'espace de recherche. Chacune d'elles définit une manière spécifique de modifier la solution courante afin d'en générer de nouvelles, on trouve par exemple :

- **2-swap** : consiste à échanger des sous-ensembles d'affectations entre deux machines ou deux positions.
- **2-opt** : remplace deux arêtes dans une solution par deux autres afin de supprimer les croisements éventuels et d'améliorer la structure.
- **3-opt** : extension du 2-OPT, il consiste à retirer trois arêtes et à reconnecter les segments de manière optimale. Il offre une exploration plus fine mais au prix d'un coût plus important.

Voisinage	**Complexité temporelle**
2-swap	$\mathcal{O}(N^2)$
2-opt	$\mathcal{O}(N^3)$
3-opt	$\mathcal{O}(N^4)$

TABLE 3 – Différents types de voisinage et leurs complexités temporelles

Nous avons fait le choix de générer du voisinage par échanges (2-swap), puisque cela permet d'explorer plus largement l'espace de recherche, avec un coût computationnel moins élevé que d'autres méthodes.

5.1.4 Pseudo-code de la méthode taboue

On génère un ensemble de voisins en effectuant des échanges de positions entre deux machines :

Algorithm 5 Génération du voisinage par échanges

Entrée: $Affectation$, *contraintes*
Sortie : Liste des voisins possibles

Voisins ← [ ] Clés ← liste des clés de $Affectation$
for $i \leftarrow 1$ **to** *longueur(clés)* **do**
  for $j \leftarrow i + 1$ **to** *longueur(clés)* **do**
    $m_1, m_2 \leftarrow$ clés[i], clés[j] $e_1, e_2 \leftarrow Affectation[m_1], Affectation[m_2]$
    if $e_2 \in contraintes[m_1]$ ***ou*** $e_1 \in contraintes[m_2]$ **then**
      continuer
    end
    $v \leftarrow$ copie de $Affectation$ échanger $v[m_1]$ et $v[m_2]$ ajouter $((m_1, m_2), v)$ à voisins
  end
end
return *voisins*

Le nombre de combinaisons est de l'ordre de $\mathcal{O}(N^2)$. Pour chaque voisin valide, la fonction de coût est recalculée. Cette évaluation a elle-même une complexité de l'ordre de $\mathcal{O}(N^2)$. Ainsi, la génération et l'évaluation complète du voisinage d'une solution demandent un temps proportionnel à $\mathcal{O}(N^4)$ dans le pire des cas.

Algorithm 6 Algorithme de recherche taboue

Entrée: AffectationInitiale, NbMaxIter, NbMaxIterStable, TabuTenure
Sortie : MeilleureSolution et coût associé

MeilleureSolution ← AffectationInitiale ListeTaboue ← [ ] IterationStable ← 0
for $i \leftarrow 1$ **to** *NbMaxIter* **do**
  Voisins ← générer tous les échanges possibles Candidats ← voisins non tabous ou meilleurs que MeilleureSolution
  if *aucun candidat* **then**
    break
  end
  MeilleurCandidat ← candidat avec le plus faible coût Ajouter le mouvement à la liste taboue
  if *taille(ListeTaboue)* > *TabuTenure* **then**
    retirer le plus ancien élément
  end
  if *coût(MeilleurCandidat)* < *coût(MeilleureSolution)* **then**
    MeilleureSolution ← MeilleurCandidat IterationStable ← 0
  end
  else
    IterationStable ← IterationStable + 1
  end
  if *IterationStable* > *NbMaxIterStable* **then**
    break
  end
end
return *MeilleureSolution et coût associé*

En outre, la sélection du meilleur candidat parmi les voisins et la mise à jour de la

liste taboue (de taille limitée T_{tenure}) introduisent un surcoût négligeable devant l'ordre de $\mathcal{O}(N^2)$, comparativement au coût de l'évaluation globale. Le processus est répété jusqu'à atteindre le nombre maximal d'itérations N_{iter} ou un nombre d'itérations stables N_{stable} sans amélioration, ce qui conduit à **une complexité totale asymptotique de** :

$$\mathcal{O}(N_{\text{iter}} \times N^4)$$

5.1.5 Résultats et influence des paramètres

L'efficacité de la méthode taboue dépend fortement du réglage de ses paramètres. Une mauvaise configuration peut conduire soit à une exploration insuffisante (solutions de faible qualité), soit à un temps de calcul excessif sans gain significatif. L'objectif est donc de trouver le bon équilibre permettant de maximiser l'optimalité tout en minimisant le temps d'exploration.

Le **nombre maximal d'itérations** détermine le nombre total d'itérations autorisées pour la recherche taboue. Si cette valeur est trop faible, la méthode risque de s'arrêter avant d'avoir exploré une portion significative de l'espace des solutions. Si elle est trop élevée, le temps de calcul augmente considérablement, sans garantie d'amélioration importante.

Le **nombre d'itérations stables** correspond au nombre d'itérations consécutives sans amélioration avant d'arrêter la recherche. Il agit comme un détecteur de stagnation. Une valeur trop basse provoque un arrêt prématuré de la recherche. Une valeur trop haute augmente inutilement la durée du calcul lorsque la solution est déjà stabilisée.

Enfin, le paramètre ayant la plus grande influence sur la solution est **la mémoire taboue** : la durée de vie d'un mouvement interdit (en nombre d'itérations). Une valeur trop faible peut autoriser trop rapidement les mouvements récents risquant ainsi la création de boucles et de stagnation. Au contraire, une valeur trop élevée peut bloquer un nombre trop important de mouvement, ce qui peut induire une perte d'intensification locale.

Ainsi, une bonne manière d'améliorer notre solution taboue consiste en le fait de modifier ces paramètres (notamment la mémoire taboue) afin d'intensifier ou diversifier la recherche et trouver des chemins accédant à de meilleures solutions.

L'analyse de sensibilité suivante sera appliquée sur la solution gloutone.

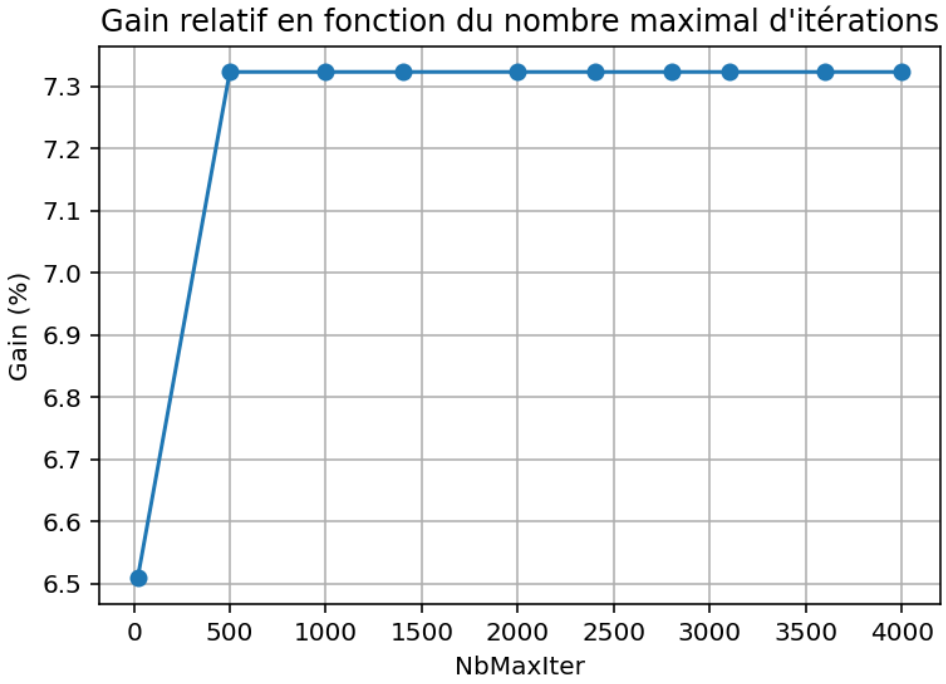


FIGURE 3 – Évolution du gain relatif en fonction du nombre maximal d'itérations, avec : $NbMaxIterStable = 200$ et $TabuTenure = 7$

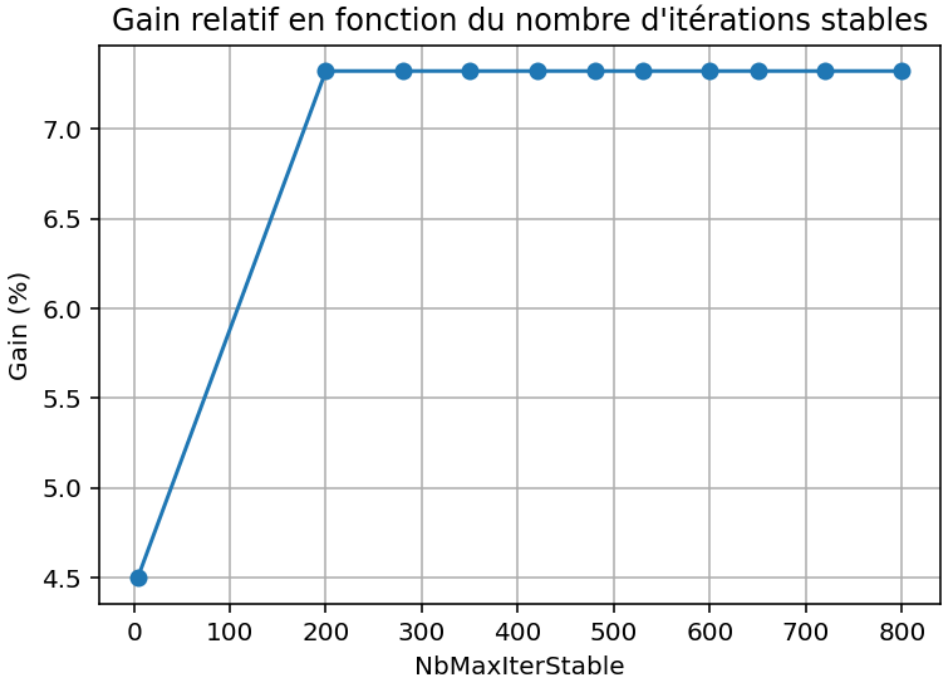


FIGURE 4 – Évolution du gain relatif en fonction du nombre d'itérations stables, avec : $NbMaxIter = 500$ et $TabuTenure = 7$

Les résultats montrent que le gain relatif augmente rapidement lors des premières itérations, puis atteint un plateau dès que **le nombre maximal d'itérations** dépasse une centaine. Au-delà de ce seuil, les améliorations deviennent négligeables et le gain reste

stable, autour de 7,3 %. Cela suggère que **la méthode taboue converge rapidement vers une solution quasi-optimale** et qu'un nombre d'itérations trop élevé n'apporte pas de bénéfice significatif, tout en augmentant inutilement le temps de calcul.

De même, le gain relatif augmente au début, puis se stabilise rapidement dès que **le nombre maximal d'itérations stables** dépasse une faible valeur. Au-delà de ce seuil, les améliorations sont insignifiantes et le gain reste constant, autour de 7.3 %. **Ce comportement est cohérent avec la dynamique de la recherche taboue**, qui atteint une solution stable après un certain nombre d'itérations sans amélioration. Ainsi, augmenter davantage le nombre d'itérations stables ne modifie pas la qualité de la solution, mais prolonge simplement le temps d'exécution.

L'analyse de l'impact de la taille de la mémoire taboue montre que le gain relatif est faible lorsque cette taille est trop petite, mais qu'il diminue également lorsque la mémoire devient trop grande.

Une mémoire taboue de faible dimension autorise trop rapidement la réutilisation de mouvements récents, ce qui favorise la création de boucles et peut entraîner une stagnation prématurée de la recherche (**manque de diversification**). À l'inverse, une mémoire trop étendue interdit un trop grand nombre de mouvements, limitant ainsi l'exploration locale et **réduisant la capacité d'intensification** de l'algorithme.

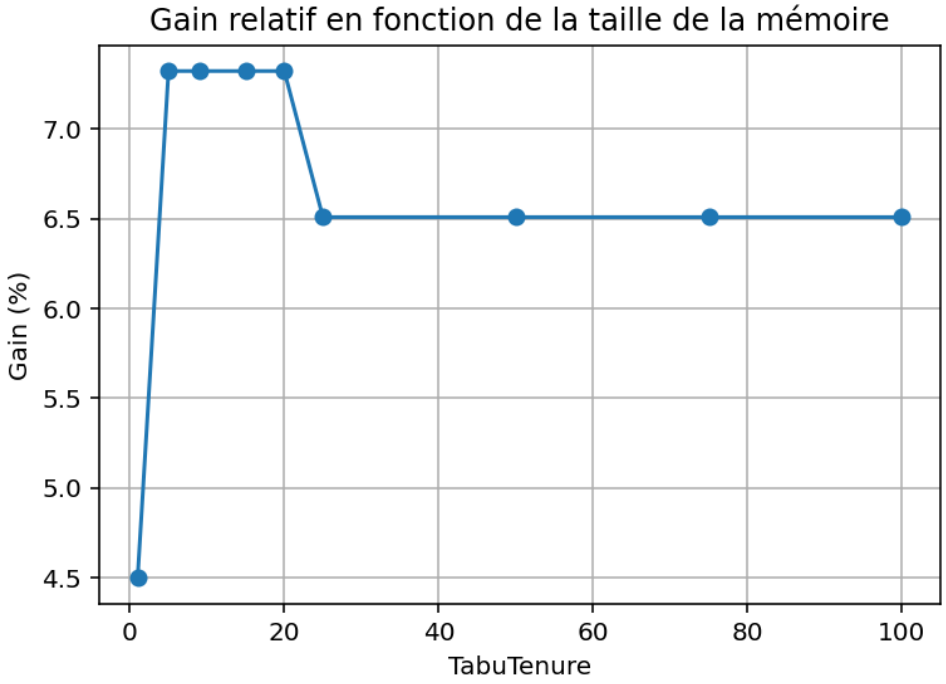


FIGURE 5 – Évolution du gain relatif en fonction de la taille de la mémoire taboue, avec : $NbMaxIter = 500$ et $NbMaxIterStable = 200$

Le meilleur compromis se situe donc pour **des valeurs intermédiaires de la taille de la mémoire taboue**, permettant un équilibre entre diversification et intensification.

5.2 Méthode du recuit simulé

5.2.1 Présentation

Le **recuit simulé** (ou *Simulated Annealing*) s'inspire du processus physique de **refroidissement des métaux**. Lorsqu'un métal est refroidi lentement, les atomes explorent plusieurs configurations avant de se stabiliser dans un état d'énergie minimale. En optimisation :

- La solution correspond à un état du système.
- La fonction de coût joue le rôle de l'énergie.
- La température contrôle la probabilité d'accepter une dégradation temporaire.
- En début de recherche, on accepte souvent des solutions moins bonnes pour explorer.
- À la fin, on devient plus sélectif pour se stabiliser sur un minimum global.

Cette méthode permet de **s'échapper des minima locaux**, contrairement à la descente locale classique.

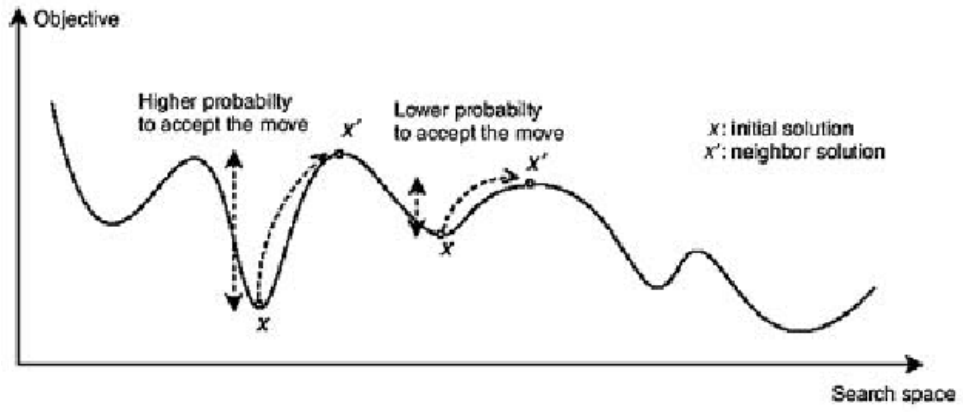


FIGURE 6 – Illustration du fonctionnement de l'algorithme de recuit simulé.

5.2.2 Principe général de la méthode

1. Initialiser une solution S_0, un coût C_0 et une température initiale T_0 élevée.
2. Tant que $T > T_{min}$:
   - Générer un voisin S' par échange aléatoire de positions entre deux machines.
   - Calculer la différence de coût $\Delta = C(S') - C(S)$.
   - Si $\Delta < 0$, accepter directement S', puisqu'il permet d'améliorer le coût.
   - Sinon, accepter S' avec une probabilité $e^{-\Delta/T}$.
   - Mettre à jour la meilleure solution et diminuer la température : $T \leftarrow \alpha \times T$.
3. Retourner la meilleure solution trouvée.

5.2.3 Pseudo-code du recuit simulé

Algorithm 7 Méthode du Recuit Simulé

Entrée: S_{init}, T_0, $T_{\min}$, facteur de refroidissement α
Sortie : Meilleure solution S_{best} et son cout C_{best}

$S \leftarrow S_{init}$ $C_S \leftarrow calculer_cot(S)$ $S_{best} \leftarrow S$ $C_{best} \leftarrow C_S$ $T \leftarrow T_0$
while $T > T_{\min}$ **do**
  for $i = 1$ **to** N_{iter} **do**
    $S' \leftarrow voisin(S)$ $C_{S'} \leftarrow calculer_cout(S')$ $\Delta \leftarrow C_{S'} - C_S$
    if $\Delta < 0$ ***ou*** $rand() < e^{-\Delta/T}$ **then**
      $S \leftarrow S'$ $C_S \leftarrow C_{S'}$
      if $C_S < C_{best}$ **then**
        $S_{best} \leftarrow S$ $C_{best} \leftarrow C_S$
      end
    end
  end
  $T \leftarrow \alpha \times T$
end
return S_{best}, C_{best}

La génération d'un voisin à chaque itération, effectuée par la fonction voisin, consiste en un échange aléatoire de deux positions dans la solution courante, ce qui s'exécute en temps constant, soit $O(1)$. Chaque itération interne du recuit (la boucle *for* de taille N_{iter} réalise ainsi une génération de voisin et une évaluation de coût, pour une complexité $O(N_{iter} \times E) = O(N_{iter} \times N^2)$ par température. Le nombre de niveaux de température K est approximativement $K = \dfrac{\log(T_{\min}/T_0)}{\log(\alpha)}$.

La **complexité totale du recuit simulé** peut donc être estimée à :

$$O(K \times N_{iter} \times N^2)$$

5.2.4 Choix des paramètres de la méthode

Le choix de la température initiale T_0 est une étape importante dans la performance du recuit simulé, car **il conditionne la capacité de l'algorithme à explorer efficacement l'espace des solutions dès le début de la recherche**. Une température trop faible conduit à un comportement de simple descente locale, tandis qu'une température trop élevée rend l'algorithme quasi aléatoire, retardant ainsi la convergence.

Dans la littérature, plusieurs méthodes ont été proposées pour déterminer une valeur initiale pertinente de T_0. L'article fondateur de Kirkpatrick et al. (1983) [1] suggère une approche empirique basée sur le taux d'acceptation des transitions défavorables. L'idée est de choisir une température telle que **la proportion de transitions acceptées au départ, notée χ_0, soit environ comprise entre 0.6 et 0.9**. Pour ce faire, on commence avec une température élevée, puis on exécute un certain nombre de transitions aléatoires. Si le taux d'acceptation observé est inférieur à la valeur cible χ_0, on multiplie

la température par 2 ; s'il est au contraire trop élevé, on la divise (par exemple par 3). Ce processus itératif est répété jusqu'à atteindre un taux d'acceptation proche de χ_0. Cette méthode, intuitive mais empirique, permet d'obtenir une température initiale cohérente avec la dynamique propre au problème traité, tout en évitant les cycles inutiles d'ajustement.

Dans le cadre de notre étude, **nous avons retenu une autre approche**, plus analytique, qui a été proposée par Johnson et al. (1989) [2]. Elle repose sur **l'estimation directe du coût moyen des transitions défavorables**. On génère aléatoirement un ensemble de transitions (voisins dont la variation de coût $\Delta > 0$) et on calcule **leur moyenne des augmentations de coût** δ. On souhaite ensuite fixer T_0 de façon à ce qu'une transition typique soit acceptée avec une probabilité χ_0. En partant de la règle d'acceptation du recuit simulé :

$$P(acceptation) = e^{-\Delta/T}$$

On obtient par inversion :

$$T_0 = -\delta/ln(\chi_0)$$

Cette méthode relie directement T_0 à la distribution empirique des différences de coût. Elle a l'avantage de fournir une estimation quantitative et reproductible de la température initiale à partir des données du problème, sans nécessiter d'essais successifs. Ben-Ameur (2004) [3] la recommande explicitement pour sa rapidité de calcul et sa précision expérimentale.

Nous avons appliqué cette approche sur un échantillon de 100 transitions aléatoires. Parmi celles-ci, 79 transitions présentaient une dégradation du coût ($\Delta > 0$) et ont donc été retenues pour le calcul. La moyenne des augmentations de coût positives observées est de $\Delta = 149,974.28$. En fixant le taux d'acceptation cible à $\chi_0 = 0.8$, la température initiale est :

$$T_0 \approx 672,097.75$$

5.2.5 Résultats et analyse

Dans le cadre du protocole expérimental, **le taux de refroidissement a été fixé à** $\alpha = 0.95$, de manière à assurer un refroidissement lent et progressif du système, sans impacter le temps d'exécution de l'algorithme. Ce choix favorise une exploration plus approfondie de l'espace des solutions en maintenant plus longtemps une probabilité significative d'acceptation des transitions défavorables, ce qui limite le risque de convergence prématurée vers un minimum local. **Le nombre d'itérations est** $N_{iter} = 500$, une valeur relativement élevée permettant de garantir une stabilisation statistique des acceptations avant chaque étape de refroidissement. Enfin, **la température minimale a été fixée à**

$T_{min} = 10^{-3}$, seuil en dessous duquel les transitions défavorables deviennent pratiquement nulles, marquant la fin du processus de recuit.

Solution	**Coût initial**	**Coût final**	**Gain total (%)**	**Temps (ms)**
Gloutonne	1 716 999	1 591 267	7.32	3042.61
Gloutonne aléatoire	1 782 218	1 558 276	12.57	3089.07

TABLE 4 – Résultats du recuit simulé sur les solutions gloutonne et gloutonne aléatoire

5.2.6 Quelques remarques importantes

Les temps d'exécution observés pour le recuit simulé, supérieurs à 3 000 ms même sur une instance d'une petite taille. Cette **durée relativement élevée peut être réduite** en ajustant certains paramètres, notamment en diminuant le nombre d'itérations N_{iter} ou la température initiale T_0, ce qui permettrait d'atteindre plus rapidement la température finale T_{min}. En revanche, le facteur de refroidissement α n'a qu'un impact marginal sur le temps d'exécution. Toutefois, réduire ces paramètres pour accélérer le calcul compromet significativement la qualité de la solution obtenue : le gain, déjà relativement modeste (pour les deux solutions), se dégrade encore lorsque la recherche est écourtée. Ainsi, **un tel ajustement représente un compromis défavorable entre temps de calcul et performance**.

Par ailleurs, l'**implémentation du recuit simulé diffère légèrement de celle présentée en cours**. Dans notre cas, pour chaque température, **un seul voisin est généré et évalué**, puis accepté ou rejeté selon le critère de Metropolis : acceptation automatique si le coût diminue, ou avec une probabilité $\exp(-\Delta/T)$ sinon. La température est ensuite immédiatement mise à jour selon la loi de refroidissement $T \leftarrow \alpha T$.

Dans la version vue en cours, un **palier de température** est imposé : pour chaque température, un nombre d'itérations par palier est fixé, ce qui permet une meilleure exploration à température constante avant de refroidir le système. Néanmoins, cette approche s'est révélée **beaucoup trop coûteuse en temps de calcul, en particulier pour la grande instance**, au point de rendre les expérimentations difficilement exploitables (temps excessifs, blocages et instabilités observées lors des tests).

C'est pourquoi nous avons opté pour une version simplifiée avec un seul voisin par température,

5.3 Algorithme génétique

5.3.1 Présentation

Un **algorithme génétique (AG)** est une méthode d'optimisation qui imite le fonctionnement de l'évolution naturelle. Ils reposent sur la génération et l'évolution d'une population de solutions candidates à travers des opérateurs tels que **la sélection, le croisement** et **la mutation**, jusqu'à ce qu'elle converge vers un résultat satisfaisant.

Chaque individu correspond à un ordre précis dans lequel les machines sont disposées dans l'atelier : l'indice correspond à la machine et la valeur à l'emplacement choisi. L'objectif est de minimiser la fonction de coût, tout en respectant certaines contraintes imposées, telles qu'une séquence qui doit toujours commencer par 0 et se terminer par 9 — symbolisant le point de départ et le point d'arrivée fixes dans le processus.

Ainsi, en appliquant cette méthode au problème d'emplacement de machines, l'AG explore efficacement un grand espace de solutions possibles, équilibrant **exploration** (diversité des solutions) et **exploitation** (amélioration progressive des meilleures configurations).

5.3.2 Principe général de la méthode

1. **Initialisation** : Une population initiale

   $$P(0) = \{x_1, x_2, \ldots, x_N\}$$

   est générée aléatoirement, où chaque individu x_i est un ordre valide respectant les contraintes.

2. **Évaluation :** Chaque individu est évalué par la fonction objectif $f(x_i)$. Cette fonction mesure la performance de la séquence

3. **Sélection :** Cette étape vise à extraire des individus parents proportionnellement à leur qualité. Nous utilisons ici une **sélection par tournoi** : un sous-ensemble de k individus est choisi aléatoirement, et le meilleur d'entre eux est retenu pour la reproduction.

   Cette approche représente un compromis efficace entre **sélectivité** et **diversité génétique** : elle favorise les individus performants sans éliminer prématurément la variabilité au sein de la population. Par rapport à d'autres méthodes telles que la sélection par roulette ou par rang, la sélection par tournoi présente plusieurs avantages : elle est plus robuste face aux écarts d'échelle de la fonction objective, plus simple à implémenter, et sa complexité est maîtrisée ($O(k)$ par sélection).

   Le choix d'**un tournoi binaire** ($k = 2$) permet de réduire le coût de calcul tout en assurant une pression sélective modérée, ce qui favorise une convergence stable sans appauvrissement rapide de la population.

4. **Croisement (Order Crossover – OX) :** Le croisement est l'opérateur clé de recombinaison. Pour deux parents p_1 et p_2, le **croisement OX (Order Crossover)** consiste à :

   (a) Sélectionner aléatoirement deux points de coupure i et j dans la séquence.

   (b) Copier la sous-séquence $p_1[i : j]$ dans le descendant.

   (c) Compléter le reste du chromosome avec les éléments de p_2 dans leur ordre d'apparition, en évitant les duplications.

Ce mécanisme préserve à la fois la structure d'ordre et la validité de la permutation.

5. **Mutation :** La mutation introduit une perturbation locale pour **préserver la diversité génétique** et éviter la convergence prématurée. Trois opérateurs sont utilisés :
   (a) **Probabilité de mutation :** La mutation n'est pas appliquée systématiquement à chaque individu. Une probabilité p_m détermine si le chromosome subira une mutation lors de son passage. Cette approche permet de limiter la déstabilisation excessive de la population tout en maintenant un minimum de diversité.
   (b) **Sélection aléatoire des positions :** Si la mutation est déclenchée, deux indices i et j sont tirés aléatoirement dans la plage $[0, n-1]$, où n est la longueur du chromosome. Ces indices correspondent à deux gènes de la séquence à échanger.
   (c) **Échange des gènes :** Les deux éléments sélectionnés sont intervertis selon la règle suivante :
   $$\text{chrom}[i], \text{chrom}[j] = \text{chrom}[j], \text{chrom}[i]$$
   (d) **Réparation éventuelle :** Après la mutation, la fonction `repair_chromosome(chrom)` est appelée afin de vérifier et corriger la validité du chromosome s'il ne respecte pas les contraintes.
6. **Élitisme et remplacement :** Afin d'assurer la préservation des meilleures solutions, un mécanisme d'élitisme est introduit : **les** $elitisme_rate$ **% meilleurs individus de la génération** t **sont automatiquement transférés dans la génération** $t+1$. Cette approche empêche toute dégradation de la qualité globale de la population et accélère la convergence.
7. **Critère d'arrêt :** L'algorithme continue de s'exécuter jusqu'au **nombre maximal de générations** défini par `generations`, mais il peut également s'arrêter plus tôt si **aucune amélioration du meilleur score n'est observée** pendant un certain nombre de générations consécutives, défini par `stagnation_limit`.

Ce fonctionnement est synthétisé sur la figure ci-dessous, qui résume les principales étapes de l'algorithme.

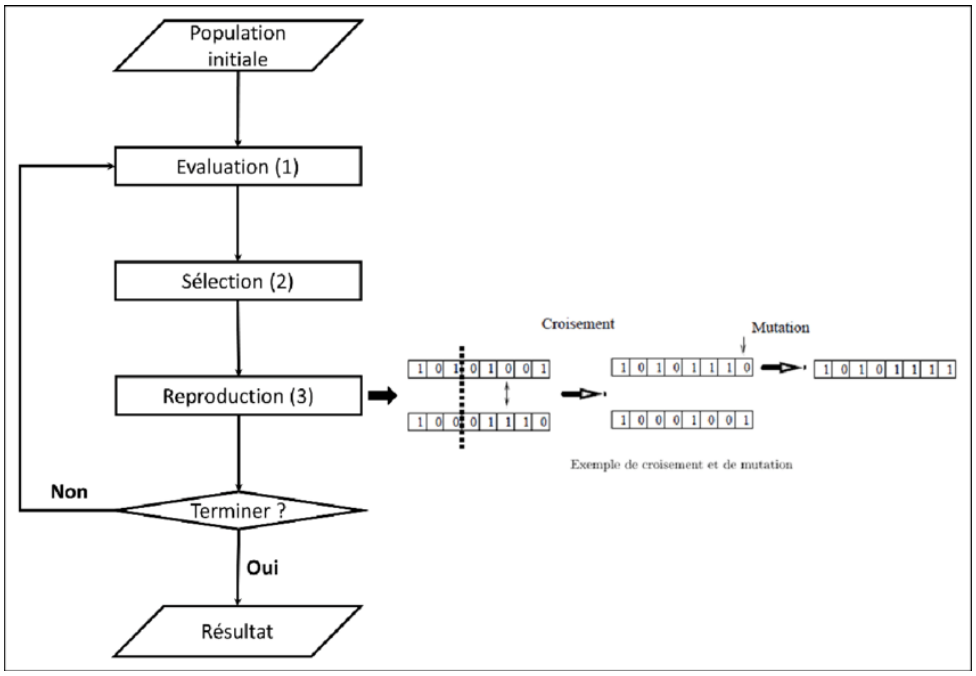


FIGURE 7 – Schéma fonctionnel de l'algorithme génétique.

5.3.3 Pseudo-code de l'algorithme génétique

Pour réparer un chromosome après une mutation ou un croisement, on commence par **identifier les doublons** et les remplacer par des éléments libres, puis **vérifier les contraintes** pour chaque gène. Si violation, il faut **échanger ou remplacer par un gène valide**.

Algorithm 8 Repair Chromosome

Entrée: *chrom*
Sortie : *chrom* réparé

used ← ensemble des gènes dans *chrom* $free \leftarrow emplacements \setminus used$
for $i = 0$ **to** *longueur(chrom)-1* **do**
  if *chrom*[i] *est un doublon* **then**
    if $free$ *non vide* **then**
      chrom[i] ← premier élément de $free$ Retirer ce gène de $free$
    end
    else
      Chercher un k tel que *chrom*[k] peut être échangé avec *chrom*[i] Si trouvé, échanger *chrom*[i] et *chrom*[k]
    end
  end
end
for $i = 0$ **to** *longueur(chrom)-1* **do**
  $m \leftarrow machines[i]$ **if** *chrom*[i] $\in$ *contraintes*[m] **then**
    Chercher un gène ej dans *chrom* qui peut être échangé avec *chrom*[i] **if** *échange impossible* **then**
      Remplacer *chrom*[i] par un gène libre valide
    end
  end
end
Vérifier à nouveau les doublons et remplacer les restants par des gènes libres
return *chrom*

Cette fonction fait une première boucle sur les N gènes pour détecter les doublons ($O(N)$). Si doublon, elle cherche des échanges possibles (pouvant aller jusqu'à $O(N)$ dans le pire cas). Une seconde boucle pour vérifier les contraintes : encore une boucle sur N avec des sous-recherches sur N. Ainsi, dans le pire des cas, **la complexité globale de *repair_ chromosome* est** $O(N^2)$.

Algorithm 9 Mutation par échange

Entrée: *chrom*, probabilité de mutation p_m
Sortie : *chrom* final

if *random()* $< p_m$ **then**
  Choisir deux indices i et j aléatoires
  Échanger *chrom*[i] et *chrom*[j]
  chrom ← RepairChromosome(*chrom*)
end
return *chrom*

La fonction mutation applique une simple mutation par échange de deux positions aléatoires dans le chromosome, ce qui est une opération élémentaire en temps constant, $O(1)$. Cependant, une fois l'échange effectué, la fonction appelle *repair_ chromosome* pour s'assurer que la solution reste valide. Ce qui donne **une complexité totale de** $O(N^2)$.

Algorithm 10 Order Crossover

Entrée: $parent1$, $parent2$
Sortie : $child1$, $child2$

Choisir deux points de coupure a et b aléatoires
for $i = a$ **to** b **do**
| $child1[i] \leftarrow parent1[i]$ $child2[i] \leftarrow parent2[i]$
end
Compléter le reste des positions de $child1$ avec les gènes manquants de $parent2$

Compléter le reste des positions de $child2$ avec les gènes manquants de $parent1$

$child1 \leftarrow$ RepairChromosome($child1$) $child2 \leftarrow$ RepairChromosome($child2$)
return $child1$, $child2$

La fonction de croisement *order_ crossover* commence par sélectionner deux points de coupure et copie la portion correspondante de chaque parent dans les enfants. Cette première partie est linéaire, de complexité $O(N)$. Cependant, le remplissage des gènes manquants nécessite des vérifications pour éviter les duplications, ce qui implique des comparaisons multiples sur des sous-listes, pouvant atteindre un coût en $O(N^2)$. De plus, les deux enfants sont ensuite réparés à l'aide de la fonction *repair_ chromosome*, donc **la complexité totale de la fonction croisement s'exprime** $O(N^2)$.

Algorithm 11 Algorithme Génétique

Entrée: SolutionInitiale, PopSize, NbGenerations, p_m, stagnation_limit, elitism_rate
Sortie : MeilleureSolution

Population ← PopulationInitiale(SolutionInitiale, PopSize)

Évaluer le coût de chaque individu de population

MeilleureSolution ← individu avec coût minimal

stagnation_counter ← 0 elitism_count ← $PopSize \times elitism_rate$

for $g = 1$ **to** *NbGenerations* **do**

  NouvellePopulation ← [ ]

  Trier population selon coût croissant

  elites ← prendre les elitism_count meilleurs individus

  NouvellePopulation ← $NouvellePopulation + elites$

  while *taille(NouvellePopulation)* $<$ *PopSize* **do**

    parent1 ← TournoiBinaire(population) parent2 ← TournoiBinaire(population) enfant1, enfant2 ← Croiser(parent1, parent2) Muter(enfant1, p_m) Muter(enfant2, p_m) Ajouter enfant1 à NouvellePopulation

    if *taille(NouvellePopulation)* $<$ *PopSize* **then**

      Ajouter enfant2 à NouvellePopulation

    end

  end

  Population ← NouvellePopulation

  Mettre à jour MeilleureSolution si un individu a meilleur coût, puis stagnation_counter

  if *stagnation_counter* $\geq$ *stagnation_limit* **then**

    Sortir de la boucle

  end

end

return *MeilleureSolution*

L'algorithme génétique complet combine les phases précédentes sur plusieurs générations. Durant l'initialisation, il crée la population de taille P avec un coût $O(P \times N^2)$ et évalue chaque individu, ce qui ajoute $O(P \times E)$. À chaque génération, il procède à la sélection des parents, au croisement, à la mutation et à la réévaluation, puis la sélection par tournoi est linéaire. Ainsi, pour G générations, **la complexité globale de l'algorithme génétique s'écrit** :

$$O(G \times P \times (N^2 + E) = O(G \times P \times N^2)$$

5.3.4 Résultats et influence des paramètres

Dans un algorithme génétique, plusieurs paramètres influencent directement la qualité et la convergence des solutions. Parmi eux, la **taille de la population** et le **taux**

d'élitisme sont particulièrement déterminants.

Une population trop petite peut conduire à une convergence prématurée, car le nombre limité d'individus réduit la diversité génétique et augmente le risque que l'algorithme reste bloqué dans un optimum local. À l'inverse, une population trop grande accroît considérablement le coût de calcul sans garantir une amélioration proportionnelle des résultats.

Le **taux d'élitisme**, qui détermine la proportion des meilleurs individus conservés d'une génération à l'autre, influence quant à lui l'équilibre entre **exploitation** et **exploration** : un taux trop faible risque de perdre des solutions prometteuses, tandis qu'un taux trop élevé peut réduire la diversité et provoquer une stagnation.

Pour ces raisons, nous avons choisi de concentrer notre analyse de sensibilité sur ces deux paramètres. **L'étude suivante sera appliquée sur la solution gloutone aléatoire**.

Les autres paramètres restent fixes pour isoler l'effet :

$$\text{NbGenerations} = 200, \quad p_m = 0.2 \quad \text{et} \quad stagnation_limit = 50$$

L'**analyse de sensibilité** en fonction de la **taille de la population** montre des fluctuations intéressantes. Pour les valeurs très basses de PopSize (par exemple 10), le gain relatif est limité. Cela s'explique par le fait qu'une population trop petite réduit fortement la diversité génétique, ce qui limite l'exploration de l'espace de solutions et augmente le risque de convergence prématurée vers des optima locaux.

En revanche, dès que la taille de la population atteint environ 50 individus, le gain relatif s'améliore significativement et atteint des valeurs comparables à celles obtenues avec des populations beaucoup plus grandes, comme 200 individus. Au-delà de 50, l'augmentation de PopSize n'apporte donc pas de gains supplémentaires en termes de qualité de solution, mais augmente le coût et le temps de calcul.

Ainsi, cette taille intermédiaire représente un compromis optimal : elle assure une diversité suffisante pour explorer efficacement l'espace de recherche tout en limitant le coût de calcul, ce qui rend l'algorithme plus efficient sans perte de performance.

Ensuite, pour de faibles taux d'élitisme (inférieurs à 10 %), le gain relatif reste bas, car l'algorithme perd trop rapidement des individus de bonne qualité, ce qui **empêche de garder les meilleures solutions trouvées**. Le gain augmente ensuite fortement entre 20 % et 40 %, zone où l'équilibre entre la conservation des bons individus et la génération de nouvelles solutions est optimal. En revanche, lorsque le taux d'élitisme est très élevé, le gain diminue à nouveau : une trop grande proportion d'individus identiques est conservée, ce qui **réduit la diversité génétique de la population** et favorise une convergence prématurée vers un optimum local.

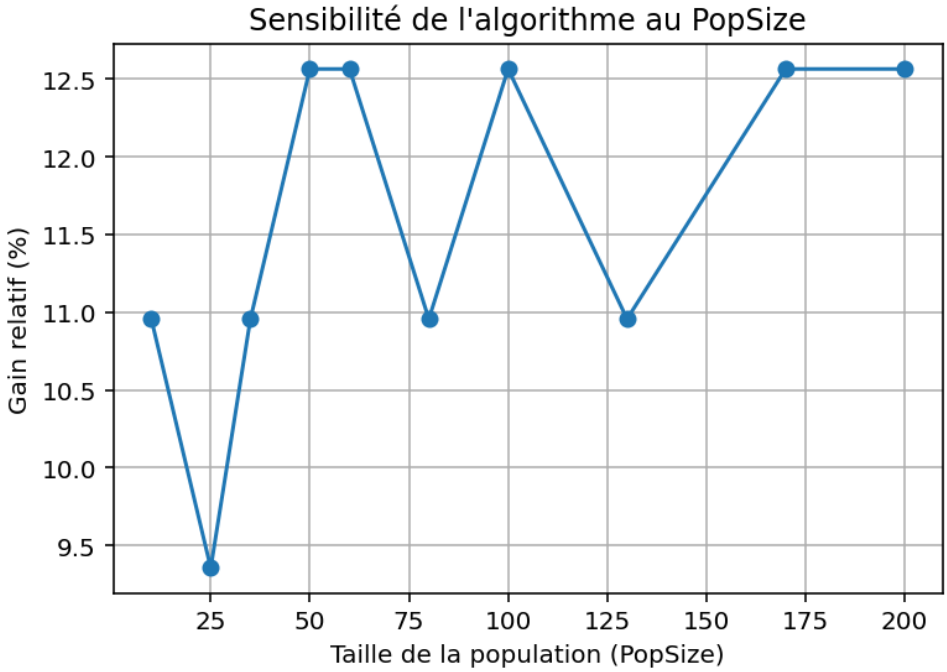


FIGURE 8 – Évolution du gain relatif en fonction de la taille de la population, avec : Taux d'élitisme = 0.2

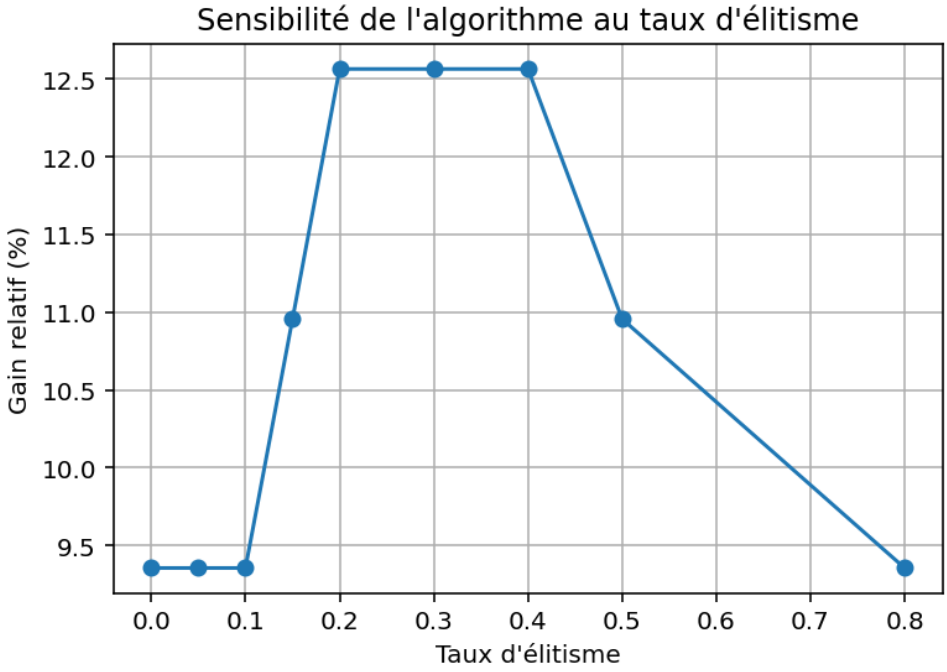


FIGURE 9 – Évolution du gain relatif en fonction du taux d'élitisme, avec : $PopSize = 50$

Dans **la littérature des algorithmes génétiques**, plusieurs travaux montrent que le taux d'élitisme est souvent calibré autour de 1 % à 20 % et rarement plus élevé pour maintenir un bon compromis entre conservation des meilleures solutions et maintien de la

diversité. Néanmoins, **une publication [4] rapporte que l'élitisme peut être réglé à « 10 %, 20 % ou 50 % de la population selon le comportement des cas de test »**. Ces valeurs justifient notre choix de viser également des valeurs supérieures à 20 % d'élitisme dans notre cas.

5.3.5 Remarque importante

La **probabilité de mutation a été fixée à 20 %, soit la borne supérieure des valeurs recommandées dans la littérature** (généralement entre 1 % et 20 %), afin de compenser le taux d'élitisme important utilisé dans l'algorithme. En effet, un élitisme fort favorise la conservation des meilleures solutions mais réduit la diversité génétique de la population, ce qui augmente le risque de convergence prématurée vers un optimum local.

En imposant un taux de mutation plus élevé, nous avons cherché à **réintroduire de la diversité dans la population**.

De plus, les tests ont montré une faible sensibilité de l'algorithme à ce paramètre entre 1 % et 20 %, ce qui justifie le choix de la borne supérieure afin de préserver la diversité.

5.4 Algorithme de la colonie de fourmis (ACO)

5.4.1 Présentation

L'**algorithme de la colonie de fourmis (ACO)** est une métaheuristique inspirée du comportement collectif des fourmis qui déposent des phéromones pour guider la recherche de chemins optimaux. Dans notre problème d'affectation des machines, chaque fourmi construit une solution en choisissant les emplacements pour chaque machine en fonction d'une combinaison de phéromones (informations apprises) et d'une heuristique (informations locales).

Elle repose sur **le dépôt et l'évaporation de phéromones simulées** afin de converger vers des chemins de coût minimal.

5.4.2 Principe général de la méthode

1. **Initialisation** : On initialise uniformément les niveaux de phéromones sur toutes les affectations possibles.
2. **Choix de l'heuristique** : L'heuristique $\eta_{m,e}$ utilisée dans l'ACO est simplement basée sur l'inverse de la distance de l'emplacement e aux points d'entrée et de sortie :

$$\eta_{m,e} = \frac{1}{d_{entre,e} + d_{e,sortie}}$$

Cette formulation favorise naturellement le choix d'emplacements proches des zones clés, car plus un emplacement est éloigné, moins il est attractif pour la construction

de la solution. Elle fournit une information locale simple mais efficace pour guider les fourmis dès le début du processus de recherche.

3. **Construction des solutions** : Chaque fourmi construit une solution complète en choisissant probabilistiquement un emplacement pour chaque machine, en évitant les contraintes d'incompatibilité. La probabilité dépend du niveau de phéromones et d'une valeur heuristique inversement proportionnelle au coût estimé.
4. **Évaluation** : On calcule le coût total de chaque solution construite.
5. **Mise à jour des phéromones** : Les phéromones s'évaporent partiellement. De plus, dans notre implémentation, nous avons choisi une stratégie de **dépôt de phéromones élitiste** : seule la meilleure fourmi de chaque itération est autorisée à déposer des phéromones.

   Ce choix permet de renforcer uniquement les affectations associées aux solutions les plus performantes, tout en évitant que des solutions de moindre qualité ne perturbent l'apprentissage collectif. Il en résulte une convergence plus rapide vers des configurations prometteuses (**plus d'intensification**).
6. **Itération** : On répète la construction et la mise à jour pendant un nombre fixé d'itérations, en conservant la meilleure solution rencontrée.

5.4.3 Pseudo-code de l'algorithme ACO

Algorithm 12 Pseudo-code de l'algorithme de colonie de fourmis

Entree: Paramètres : n_{fourmis}, n_{iter}, α, β, ρ, Q
Sortie: Meilleure affectation *best_assign* et coût associé *best_cost*

Initialiser uniformément la matrice des phéromones $\tau_{m,e}$ pour chaque arête (m, e)

for *itération* $t = 1$ **to** n_{iter} **do**
  for *chaque fourmi* $k = 1$ **to** $n_{fourmis}$ **do**
    Initialiser une affectation vide et la liste des emplacements disponibles
    for *chaque machine* m **do**
      Identifier les emplacements candidats non incompatibles Pour chaque candidat e, calculer l'attractivité heuristique :

$$\eta_{m,e} = \frac{1}{d_{entre,e} + d_{e,sortie}}$$

      Calculer la probabilité de choisir chaque emplacement e :

$$p_{m,e} = \frac{(\tau_{m,e})^{\alpha}(\eta_{m,e})^{\beta}}{\sum_{e' \in \text{candidats}} (\tau_{m,e'})^{\alpha}(\eta_{m,e'})^{\beta}}$$

      Sélectionner un emplacement e selon la loi de probabilité $p_{m,e}$ et l'affecter à la machine m Retirer e des emplacements disponibles
    end
    Compléter l'affectation si nécessaire et calculer le coût total de la solution
    Mettre à jour la meilleure solution locale si le coût est plus faible
  end
  Mise à jour des phéromones :
  for *chaque couple* (m, e) **do**
    Appliquer l'évaporation : $\tau_{m,e} \leftarrow (1 - \rho)\,\tau_{m,e}$
  end
  Identifier la meilleure fourmi de l'itération :

$$k^* = \arg\min_k \text{coût}(\text{solution}_k)$$

  for *chaque affectation* (m, e) *appartenant à la solution de la fourmi* k^* **do**

$$\tau_{m,e} \leftarrow \tau_{m,e} + \frac{Q}{\text{coût}(\text{solution}_{k^*})}$$

  end
  Mettre à jour la meilleure solution globale *best_assign* si un meilleur coût est trouvé
end

Retourner *best_assign* et son coût *best_cost* ;

Pour construire une solution, l'algorithme parcourt toutes les N machines, pour chaque machine il considère en moyenne $O(N)$ candidats, calcule pour chaque candidat la heu-

ristique $O(1)$. Ainsi, **la complexité de la construction d'une solution est** $O(N^2)$.

Dans une itération on exécute n_{fourmis} fourmis, chacune coûtant $O(N^2)$, puis la mise à jour des phéromones, de complexité $O(N^2 + N \times n_{\text{fourmis}})$. Ainsi, le coût d'une itération est $O((n_{\text{fourmis}} + 1) \times N^2 + n_{\text{fourmis}} \times N)$. Enfin, **la complexité totale de l'algorithme ACO est** :

$$O(n_{\text{iter}} \times n_{\text{fourmis}} \times N^2)$$

5.4.4 Paramètres ACO : Rôle et interprétation

1. **Poids de la phéromone** α : contrôle la propension des fourmis à suivre les traces laissées précédemment (**exploitation**). En augmentant α on favorise l'exploitation des chemins fortement marqués ; en la diminuant on laisse plus de place à la composante heuristique (**exploration**). Le réglage de α modifie fondamentalement l'équilibre exploration/exploitation et affecte la vitesse de convergence et le risque de convergence prématurée.
2. **Taux d'évaporation des phéromones** ρ : correspond quant à lui à la vitesse de disparition des phéromones, Un ρ faible (peu d'évaporation) fait persister les phéromones : rapide convergence, risque de blocage ; un ρ élevé favorise la désapprentissage et la ré-exploration, puisque les traces s'effacent vite.
3. **Nombre de fourmis** $n_{\textbf{fourmis}}$: contrôle la largeur d'exploration par itération. Interprétation : augmenter n_{fourmis} réduit la variance des solutions par itération et augmente la couverture de l'espace, mais augmente linéairement la complexité algorithmique.
4. **Paramètre heuristique** β : multiplie l'influence de l'attractivité heuristique : le diminuer permet alors de tester des solutions qui ne semblent pas optimales de part l'augmentation de la distance mais qui à terme peuvent le devenir, tandis qu'un β trop élevé rend le comportement des fourmis trop déterministe.
5. **Quantité** Q **et initialisation des phéromones** τ_0 : l'initialisation des phéromones τ_0 influence la diversité initiale.

5.4.5 Justification du choix des paramètres à tester

Dans l'algorithme ACO, **le poids de la phéromone** α **et taux d'évaporation des phéromones** ρ **sont des paramètres fondamentaux et intimement couplés** : α règle l'effet des traces collectives (i.e. la force relative de l'exploitation des chemins marqués), tandis que ρ commande la durée de vie de cette mémoire collective. Étudier ces deux paramètres ensemble est donc justifié parce que leur interaction influence la recherche globale.

Concrètement, augmenter α renforce la probabilité que les fourmis suivent un chemin déjà fréquenté ; si, simultanément, ρ est très faible (phéromones persistantes), l'algorithme

peut converger très vite vers une solution qui était seulement « bonne au début » et manquer d'exploration. À l'inverse, avec un ρ élevé (évaporation rapide) un α fort devient inefficace parce que les traces n'ont pas le temps de s'accumuler : l'algorithme revient alors vers un comportement proche d'une recherche heuristique stochastique sans apprentissage collectif. Ces mécanismes sont décrits et analysés à la fois dans l'ouvrage de référence sur ACO [5], qui montre l'importance de la calibration croisée des paramètres, et dans des travaux qui analysent empiriquement la sensibilité de l'algorithme à ces réglages, comme par exemple dans l'article [6] ; ils recommandent explicitement de balayer α et ρ conjointement plutôt que séparément.

Par ailleurs, **des variantes performantes d'ACO (par ex. MAX–MIN Ant System — MMAS)** ont été développées pour contrôler les effets néfastes d'un mauvais couplage (α,ρ) : MMAS impose des bornes sur les valeurs de phéromone et/ou limite le dépôt pour éviter la saturation et le blocage.

Ces travaux expérimentaux servent de base méthodologique : on teste d'abord une grille (α,ρ) large et on repère des régions stables ensuite.

Le paramètre β pondère l'importance relative de la composante heuristique $\eta_{m,e}$, c'est-à-dire l'attractivité d'un déplacement en fonction du coût ou de la distance. Selon les résultats empiriques de Angus et Hendtlass (2005) [7] sur le problème du voyageur de commerce (TSP), **la plage optimale de β se situe généralement entre 5 et 12, avec un pic de performance autour de 10**. Ces observations ont été confirmées par des études ultérieures sur des variantes de l'ACO appliquées à des problèmes de planification et d'ordonnancement (Socha & Dorigo (2008) [8] et Dorian Gaertner & Keith L. Clark (2005) [9]).

Enfin, la valeur initiale des phéromones τ_0 joue un rôle uniquement lors des premières itérations. Plusieurs études (Dorian Gaertner & Keith L. Clark (2005) [9]) recommandent de **fixer τ_0 à une valeur uniforme modérée (typiquement $\tau_0 = 1.0$)**, permettant à l'algorithme de démarrer sans biais.

5.4.6 Résultats et influence des paramètres

Les résultats que nous avons trouvés montrent que les zones de gain élevé (couleurs chaudes) se concentrent pour des ρ faibles à modérés (0.01 à 0.2) et des α modérés à élevés (1.0 à 2.0). Par contre, es ρ élevés ($\geq$ 0.3–0.4) donnent des gains inférieurs. Il existe parfois un pic local qui maximise le gain moyen ($\approx$ 13% sur la carte).

Ces motifs correspondent exactement au compromis théorique exploration/exploitation décrit dans la partie précédente : α contrôle la confiance dans les phéromones, ρ contrôle la durée de mémoire collective et leur interaction influence la dynamique de convergence.

De plus, le nombre de fourmis est fixé à $n_{\text{fourmis}} = 30$, afin de limiter le temps de calcul total pour l'évaluation de toutes les combinaisons de valeurs de α et ρ, puisque chaque exécution de l'algorithme peut déjà être relativement longue.

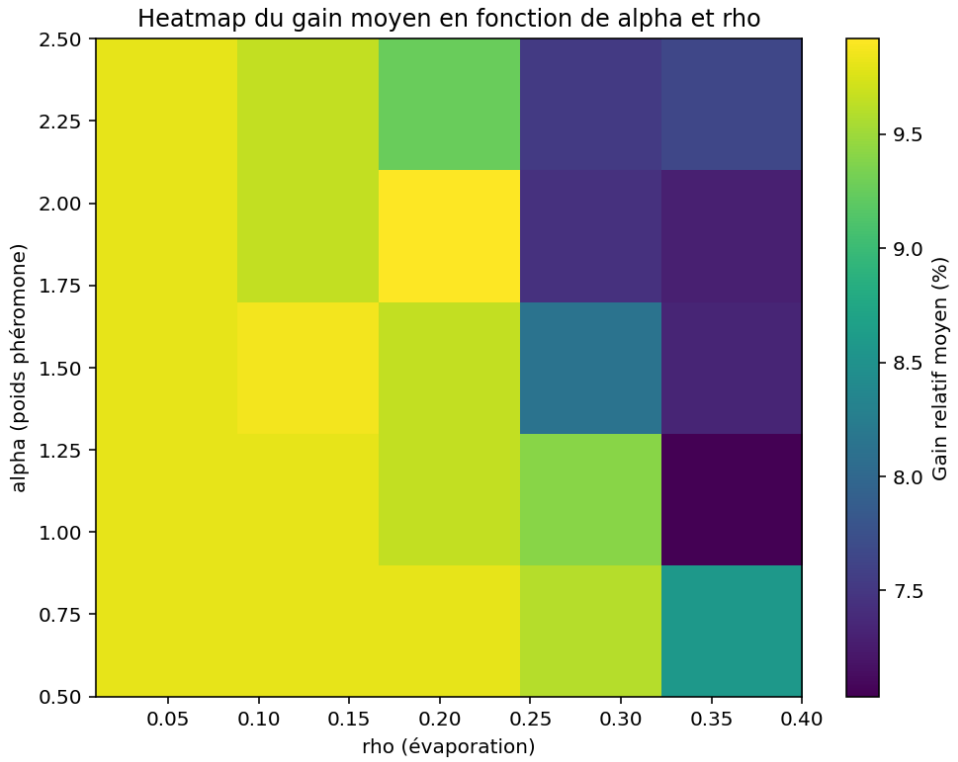


FIGURE 10 – Carte du gain moyen en fonction de α et ρ pour la solution gloutonne, avec : $n_{\text{fourmis}} = 30$, $n_{\text{iter}} = 50$ et $Q = 100$

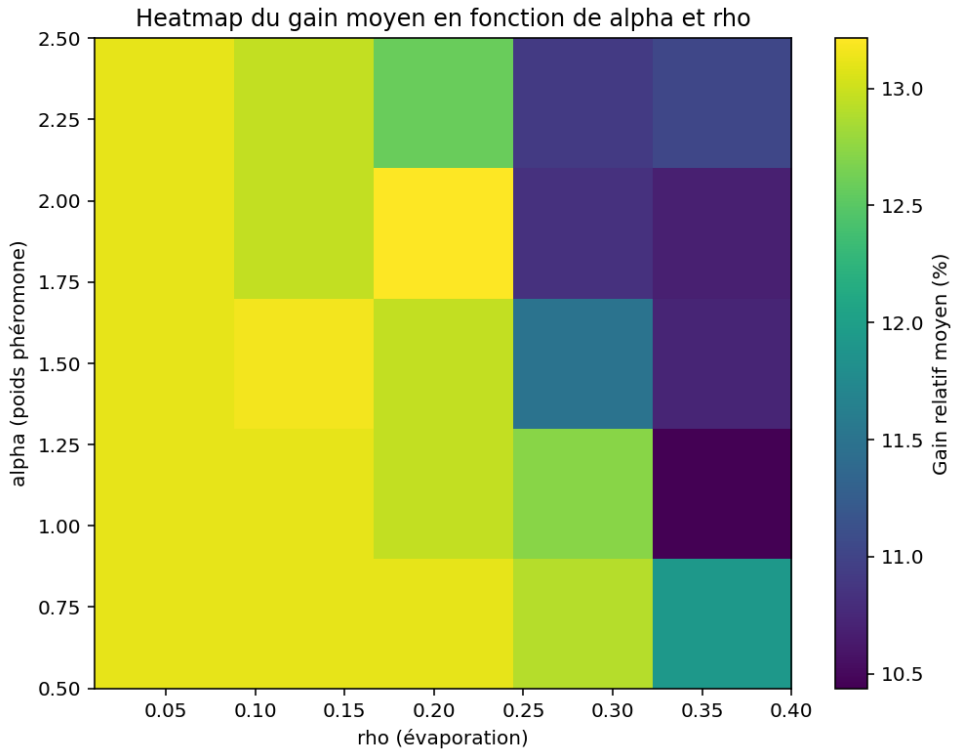


FIGURE 11 – Gain moyen en fonction de α et ρ pour la solution gloutonne aléatoire, avec : $n_{\text{fourmis}} = 30$, $n_{\text{iter}} = 50$ et $Q = 100$

Ensuite, après avoir fixé les paramètres $\alpha = 1$ et $\rho = 0.05$, nous allons expliquer en quoi **le nombre de fourmis** $n_{\textbf{fourmis}}$ constitue un paramètre clé dans l'algorithme de colonie de fourmis (ACO). En effet, il détermine la diversité de la recherche à chaque itération. Théoriquement, **un faible nombre de fourmis conduit à une exploration limitée de l'espace de solutions**. À l'inverse, **un nombre très élevé de fourmis permet une exploration beaucoup plus large**, mais au prix d'un temps de calcul plus long et d'une convergence plus lente en raison de la dispersion excessive des traces de phéromones.

Les résultats illustrés par la courbe ci-dessous montrent cette tendance générale. Pour des petites colonies (5 à 15 fourmis), le gain reste modéré, avec une amélioration inférieure à 12 %. Cela s'explique par le manque d'exploration et la forte influence du hasard dans le choix des trajectoires. Lorsque le nombre de fourmis augmente (autour de 50 à 70), on observe une hausse significative du gain, **dépassant pour la première fois 17 % d'amélioration par rapport à la solution gloutonne aléatoire**. À ce stade, l'équilibre entre diversité de recherche et intensité de convergence semble optimal. Cette solution a permis d'**obtenir un coût total d'environ 1,47 million**, soit le meilleur résultat observé sur l'ensemble des configurations testées.

Au-delà de 110 fourmis, le gain tend à se stabiliser, voire à fluctuer légèrement, sans amélioration notable. Cette saturation traduit le fait qu'au-delà d'un certain seuil, l'ajout de fourmis supplémentaires n'apporte plus de bénéfice significatif à la recherche, les informations de phéromone devenant redondantes.

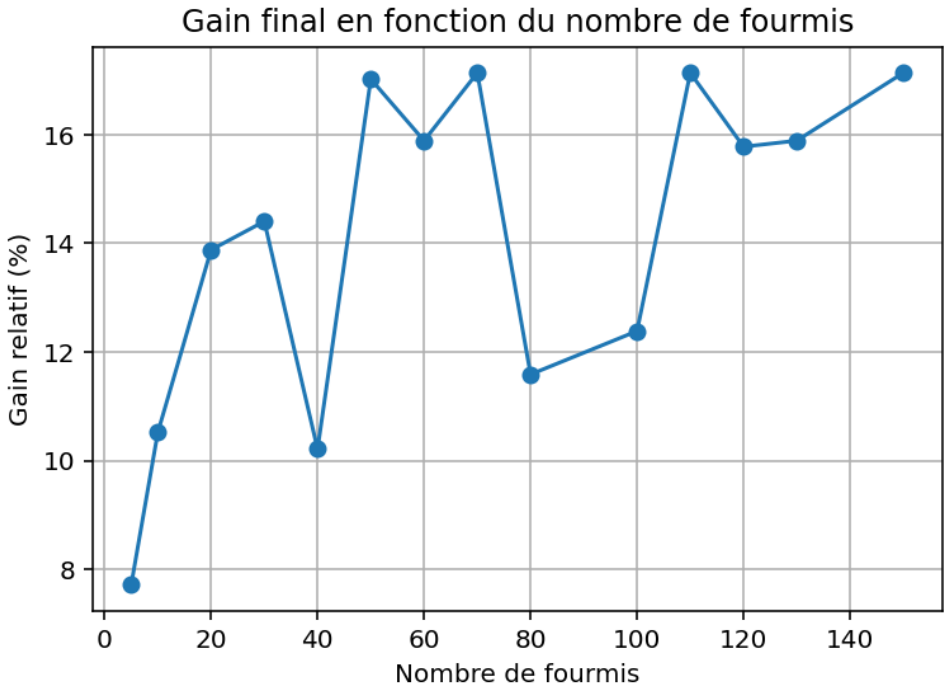


FIGURE 12 – Gain relatif en fonction de n_{fourmis} pour la solution gloutonne aléatoire, avec : $n_{\text{iter}} = 50$, $\alpha = 1$, $\rho = 0.05$ et $Q = 100$

Au total, le meilleur compromis est atteint autour de 70 fourmis, permettant d'obtenir un gain maximal et un coût optimal.

6 Gestion de la diversité dans l'algorithme génétique

Afin de prévenir la convergence prématurée de l'algorithme génétique et d'éviter que la population ne stagne dans des optimums locaux, nous avons implémenté un mécanisme de gestion de la diversité dans les populations. Ce mécanisme repose sur deux composantes principales : la gestion des doublons via la distance de Hamming et la sélection des enfants basée sur un *Goodness Score*.

6.1 Distance de Hamming

La distance de Hamming est utilisée pour quantifier la différence entre deux solutions (chromosomes). Dans le contexte de notre problème d'affectation, elle mesure le nombre d'emplacements où l'affectation diffère entre deux individus.

Soient deux chromosomes C_1 et C_2 de longueur N, où $C_x[i]$ représente l'emplacement affecté à la machine i dans le chromosome x. La distance de Hamming d_H est définie par :

$$d_H(C_1, C_2) = \sum_{i=1}^{N} \mathbb{1}(C_1[i] \neq C_2[i]) \tag{1}$$

où $\mathbb{1}$ est la fonction indicatrice qui vaut 1 si la condition est vraie et 0 sinon. Cette métrique est utilisée à deux niveaux :

— **Filtrage des doublons :** Avant d'intégrer un enfant dans la population, nous vérifions que $d_H(\text{enfant}, \text{individu}) \neq 0$ pour tout individu de la population actuelle.

— **Calcul de la diversité :** Elle sert de base au calcul de la composante de diversité dans le Goodness Score.

6.2 Goodness Score (GS)

Pour sélectionner les individus à conserver dans la nouvelle population, nous avons introduit une métrique présentée dans le cours appelée *Goodness Score*. Ce score permet d'évaluer la qualité d'un individu en pondérant deux critères : l'exploitation et l'exploration.

Pour un individu enfant c, le score est calculé comme suit :

$$GS(c) = \delta \cdot \underbrace{\left(\frac{f_{max} - f(c)}{f_{max} - f_{min} + 1} \right)}_{\text{Composante Fitness}} + (1 - \delta) \cdot \underbrace{\left(\frac{D_{min}(c) - D_{min}^{pop}}{D_{max}^{pop} - D_{min}^{pop} + 1} \right)}_{\text{Composante Diversité}} \tag{2}$$

Où :

— $f(c)$ est le coût total de l'enfant c.

— f_{max} et f_{min} sont respectivement les coûts maximum et minimum observés dans la population actuelle.

— $D_{min}(c)$ est la distance de Hamming minimale entre l'enfant c et tous les autres individus de la population ($D_{min}(c) = \min_{p \in P} d_H(c, p)$).

— D_{max}^{pop} et D_{min}^{pop} sont les bornes des distances observées dans la population.

— δ est le facteur de pondération (fixé à 0.5 dans nos expériences).

Ce score favorise les individus qui ont à la fois un coût faible et une structure différente des solutions déjà présentes.

6.3 Impact sur la qualité de la solution

Nous avons testé l'impact de l'intégration de la gestion stricte des doublons et du *Goodness Score* sur les instances du problème. L'objectif était de vérifier si le maintien forcé de la diversité permettait d'atteindre de meilleures solutions finales.

Malgré la sophistication théorique de l'approche, les résultats expérimentaux montrent que l'ajout de ces mécanismes n'a apporté **aucun gain supplémentaire** sur la qualité de la solution finale.

En comparant les résultats avec l'algorithme génétique standard (sans GS) et les solutions initiales, la valeur de la fonction objectif reste identique. Le mécanisme de diversification, bien que fonctionnel, **ne permet pas à l'algorithme de découvrir des zones plus prometteuses que celles explorées par l'algorithme génétique standard sans GS**.

6.4 Impact sur le temps de calcul

Le calcul du *Goodness Score* nécessite de calculer la distance de Hamming entre chaque candidat et l'ensemble de la population à chaque génération, introduisant une complexité quadratique par rapport à la taille de la population.

Sur la petite instance, nous observons une augmentation du temps de calcul jusqu'à l'ordre de **30 secondes** par exécution. Ce surcoût est disproportionné par rapport à l'absence de gain en qualité.

En conclusion, pour ce problème d'affectation spécifique, le ratio coût/bénéfice du *Goodness Score* est défavorable. Une stratégie de diversification plus légère, **en supprimant uniquement les doublons à l'aide de la distance de Hamming**, est donc plus pertinente.

7 Analyse et comparaison des résultats

7.1 Jeu de résultats obtenu pour la petite instance

Les différentes méthodes heuristiques et métaheuristiques ont été testées tout d'abord sur la petite instance du problème de réorganisation d'atelier, en partant de **la solution gloutonne** (page 7, coût = 1 716 999) et de **la solution gloutonne aléatoire** détaillée précédemment (page 8, coût = 1 782 860).

Les tableaux suivants présentent les résultats principaux obtenus pour chaque approche. Le premier en partant de la solution gloutonne classique et le deuxième partant de la solution gloutonne aléatoire :

Méthode (gloutonne classique)	**Coût final**	**Gain (%)**	**Temps d'exécution (ms)**
Descente locale	1 712 860	+0.24	0.8
Descente plus forte pente	1,608,464	+6.32	7.7
Méthode taboue	1 591 267	+7.32	28.7
Recuit simulé	1 591 267	+7.32	3023.21
Algorithme génétique	1 617 971	+5.77	148.8
Algorithme de fourmis	1 512 563	+11.91	75.0

Méthode (gloutonne aléatoire)	**Coût final**	**Gain (%)**	**Temps d'exécution (ms)**
Descente locale	1 586 943	+10.96	1.6
Descente plus forte pente	1,517,454	+14.86	7.5
Méthode taboue	1 558 276	+12.57	20.9
Recuit simulé	1 558 276	+12.57	2978.19
Algorithme génétique	1 558 276	+12.57	125.6
Algorithme de fourmis	**1 478 868**	**+17.02**	147.3

En complément, une campagne de **multi-initialisations** (5 départs gloutons aléatoires) a été réalisée afin d'évaluer **la stabilité et la robustesse** des différentes méthodes. Cette dernière applique la technique du parachutage en prenant comme solutions initiales cinq solutions gloutonnes aléatoires, afin d'obtenir tout autant de points de départ.

Le tableau suivant résume les statistiques obtenues :

Méthode	**Meilleur coût**	**Pire coût**	**Moyenne**	**Écart-type**	**CV (%)**
Descente locale	1 524 277	1 661 248	1 583 165	53 419	3,37
Descente plus forte pente	**1 478 868**	1 588 443	1 525 426	46 246	3,03
Méthode taboue	1 492 661	1 591 267	1 534 876	40 864	2,66
Recuit simulé	1 492 661	1 617 971	1 530 145	46 058	3,01
Algorithme génétique	1 492 661	1 617 971	1 565 478	40 528	2,59
Algorithme de fourmis	1 557 134	1 613 287	1 586 012	17 807	1,12

CV = Coefficient de Variation (CV = (Écart-type/Moyenne) × 100), permettant de mesurer la **robustesse** d'une méthode : plus le CV est faible, plus la méthode est stable.

7.2 Analyse des performances pour la petite instance

Les résultats montrent des comportements distincts selon les familles d'algorithmes :

— **La descente locale** ne permet qu'une amélioration limitée en partant de la solution gloutonne (+0,24 %) mais s'exécute quasi instantanément (< 1 ms). Elle reste alors piégée dans un minima local par son fonctionnement purement déterministe. Sur le départ glouton *aléatoire*, en revanche, le gain grimpe à +10,96 % , la solution initiale étant plus éloignée de l'optimum. Cela montre la sensibilité de cette méthode au point de départ choisi, et donc sa fragilité.

— **La méthode de descente à plus forte pente.** offre une amélioration nettement plus important que la descente locale en partant de la solution gloutonne, avec un gain atteignant +6,32 %. Ce gain monte à **+14,86 %** avec le départ glouton aléatoire, pour un temps d'exécution très court (∼7–8 ms). En explorant tous les déplacements et en prenant systématiquement le meilleur, la méthode trouve des optima locaux plus profonds, lui permettant ainsi d'atteindre un meilleur optimum local. Cependant, elle reste toutefois dépendante du point de départ, mais permet d'obtenir le **meilleur coût global** en multi-initialisations (**1 478 868**).

— **La méthode taboue** constitue un excellent compromis qualité/temps, avec des gains de +7,32 % pour le départ glouton et de +12,57 % pour le départ glouton aléatoire en ∼21–29 ms. L'introduction d'une mémoire des mouvements interdits permet de s'extraire des minima locaux, rendant la méthode plus robuste (CV ≈ 2,66 %) et efficace.

— **Le recuit simulé** permet d'obtenir des gains de +4,50 % et +12,57 % mais pour des temps sensiblement plus élevés (jusqu'à ∼2 575 ms selon le départ). Sa performance dépend fortement du réglage des paramètres (température initiale, taux de refroidissement, nombre d'itérations), mais il reste globalement stable, avec une robustesse comparable à la descente à plus forte pente (CV ≈ 3,01 %), avec le même *meilleur coût* multi-init que la taboue (1 492 661).

— **L'algorithme génétique** atteint un gain de +5,77 % depuis la gloutonne et +12,57 % depuis la gloutonne aléatoire pour des temps courts (125–150 ms). Ses performances sont proches du recuit simulé, mais pour un temps d'exécution bien plus court. Il offre une bonne robustesse (CV 2,6 %) sans toutefois égaler la méthode taboue ou la plus forte pente en termes de coût.

— **L'algorithme de colonies de fourmis (ACO)** fournit la **meilleure solution ponctuelle depuis le départ glouton** (1 512 563, +11,91 %) et également la **meilleure solution ponctuelle depuis le départ glouton aléatoire** (1 478 868

, +17,02 %). L'exécution de cette méthode s'effectue dans des temps modérés (∼75–147 ms). Enfin, cette méthode est particulièrement stable et **robuste** en multi-initialisations avec un CV ≈ 1,12 %, le plus faible, mais sans atteindre le meilleur coût global de la plus forte pente.

En synthèse, pour des exécutions depuis un départ glouton classique ou glouton aléatoire, **la méthode des colonies de fourmis** domine. Avec *multi-initialisations*, la plus forte pente s'impose (meilleur coût global), tandis que **la taboue** fournit un excellent compromis qualité/temps/robustesse et que **ACO** se distingue par sa *stabilité* (CV minimal).

7.3 Jeu de résultats obtenu pour la grande instance

Dans le domaine industriel, le jeu de données à traiter est généralement plus dense que celui sur lequel nous avons travaillé jusqu'à présent. C'est pourquoi nous allons désormais traiter une instance de taille plus importante (28 machines à placer) afin de tester le comportement de chacune de nos méthodes face à ce changement.

Pour cela, nous procéderons de manière identique à la plus petite instance, débutant par la génération d'une solution gloutonne classique et d'une solution gloutonne aléatoire dont voici les expressions :

```
=== SOLUTIONS INITIALES ===

Gloutonne classique  → Coût = 641,929,193 | Temps = 0.2 ms
Affectations :
 M1 → 26
 M2 → 20
 M3 → 1
 M4 → 5
 M5 → 2
 M6 → 6
 M7 → 7
 M8 → 8
 M9 → 9
 M10 → 10
 M11 → 11
 M12 → 17
 M13 → 12
 M14 → 28
 M15 → 14
 M16 → 13
 M17 → 22
```

```
 M18 → 15
 M19 → 16
 M20 → 18
 M21 → 19
 M22 → 21
 M23 → 3
 M24 → 23
 M25 → 24
 M26 → 25
 M27 → 27
 M28 → 4

Gloutonne aléatoire  → Coût = 615,843,219 | Temps = 0.3 ms
Affectations :
 M1 → 5
 M2 → 10
 M3 → 1
 M4 → 2
 M5 → 18
 M6 → 3
 M7 → 4
 M8 → 6
 M9 → 7
 M10 → 8
 M11 → 9
 M12 → 28
 M13 → 12
 M14 → 11
 M15 → 26
 M16 → 13
 M17 → 24
 M18 → 15
 M19 → 16
 M20 → 19
 M21 → 17
 M22 → 20
 M23 → 27
 M24 → 21
 M25 → 22
 M26 → 23
```

```
M27 → 25
M28 → 14
```

On observe ici que, contrairement à la plus petite instance, la solution gloutonne aléatoire est meilleure que celle classique, montrant l'importance de la diversité dans l'exploration de solutions pour ne pas s'enfoncer trop vite dans un optimum local peu intéressant. Les deux méthodes s'exécutent, comme l'on pourrait s'y attendre, très rapidement, dû au peu de manipulations effectuées au sein de chacune d'entre elles.

Nous avons, de la même manière que précédemment, appliqué nos méthodes heuristiques et métaheuristiques à ces deux solutions initiales pour étudier leur comportement et leur efficacité.

Départ : solution gloutonne classique (coût initial = 641,929,193**).**

Méthode (gloutonne classique)	**Coût final**	**Gain (%)**	**Temps d'exécution (ms)**
Descente locale	457 338 914	+28.76	193.5
Descente plus forte pente	470 532 716	+26.70	834.6
Méthode taboue	438 279 021	+31.72	2007.4
Recuit simulé	478 326 019	+25.49	7243.3
Algorithme génétique	472 798 078	+26.35	1663.1
Algorithme de fourmis	601 692 242	+6.27	1036.6

Départ : solution gloutonne aléatoire (coût initial = 615,843,219**).**

Méthode (gloutonne aléatoire)	**Coût final**	**Gain (%)**	**Temps d'exécution (ms)**
Descente locale	481 185 531	+21.87	228.7
Descente plus forte pente	473 961 935	+23.04	905.1
Méthode taboue	447 946 739	+27.26	2081.7
Recuit simulé	477 602 990	+22.45	7440.7
Algorithme génétique	505 300 503	+17.93	804.9
Algorithme de fourmis	608 438 474	+1.20	996.6

De même, nous avons employé la technique du parachutage en appliquant chacune de nos méthodes à cinq solutions gloutonnes aléatoires afin de tester leur stabilité et robustesse. Le tableau suivant résume les statistiques obtenues :

Méthode	Meilleur coût	Pire coût	Moyenne	Écart-type	CV (%)
Descente locale	452 768 803	488 797 198	477 258 730	12 576 295	2,63
Descente plus forte pente	449 228 636	485 955 769	469 376 793	13 050 525	2,78
Méthode taboue	446 904 169	461 142 372	454 497 027	5 384 569	1,19
Recuit simulé	440 314 372	468 767 144	459 678 075	9 955 996	2,17
Algorithme génétique	480 284 343	506 525 256	491 482 275	10 743 552	2,19
Algorithme de fourmis	596 266 480	609 971 989	604 856 851	4 986 522	0,82

CV = Coefficient de Variation (CV = (Écart-type/Moyenne) $\times$ 100), permettant de mesurer la **robustesse** d'une méthode : plus le CV est faible, plus la méthode est stable.

Au final, on obtient un meilleur coût global de **449 228 636**.

7.4 Analyse des performances pour la grande instance

Les résultats obtenus sur la grande instance mettent en évidence des différences de comportement plus marquées entre les différentes méthodes. L'augmentation du nombre de machines (28 au lieu de 8) augmente significativement la complexité du problème, élargissant l'espace de recherche et rendant la convergence vers un optimum global plus difficile.

— **La descente locale** permet ici une très grande amélioration par rapport à la solution gloutonne, avec un gain de +28,76 % pour un temps d'exécution très court (190 ms). Cette amélioration, bien plus importante que sur la petite instance, s'explique par la grande taille du problème, permettant un nombre plus important d'échanges améliorants cette solution. En revanche, comme attendu, elle reste piégée dans des minima locaux et montre une dépendance forte au point de départ, avec un écart-type de plus de 12 millions et un CV 2,63 %, traduisant une robustesse moyenne.

— **La descente à plus forte pente** confirme son efficacité avec des gains de +26,70 % (départ glouton) et +23,04 % (départ glouton aléatoire), pour un temps d'exécution inférieur à la seconde (835–900 ms). En explorant l'ensemble des déplacements possibles et en choisissant systématiquement le meilleur, elle atteint des optima locaux plus profonds que la méthode de descente locale. En multi-initialisations, elle obtient le **meilleur coût global**, à savoir 449 228 636, ce qui en fait la méthode la plus performante en termes de coût strict, bien qu'un peu moins stable (CV 2,78 %).

— **La méthode taboue** reste l'une des plus intéressantes, avec des gains très conséquents de +31,72 % (glouton) et +27,26 % (glouton aléatoire) pour un temps d'exécution correct (2 000 ms). De même que précédement, c'est la mémoire des déplacements interdits qui va permettre à cette méthode de s'extraire efficacement des minima locaux, renforçant la stabilité de son exploration. Son autre point fort est **sa grande robustesse et donc excellente efficacité** (CV 1,19 %) quel que soit le point de départ, comme

vu avec la campagne de multi-initialisation. Elle représente le meilleur compromis global entre qualité, rapidité et fiabilité.

— **Le recuit simulé** offre des gains importants (+25,49 % et +22,45 %) mais pour des temps d'exécution bien plus élevés (jusqu'à 7,4 secondes). Bien que cette méthode parvienne à éviter certains minima locaux grâce à son acceptation probabiliste de solutions moins interessantes, ses performances dépendent toujours fortement du réglage des paramètres (température initiale, facteur de refroidissement, nombre d'itérations).

— **L'algorithme génétique** conserve une bonne qualité de solution avec des gains de +26,35 % et +17,93 %, pour des temps d'exécution courts (0,8–1,6 s). Bien qu'il reste plus rapide que le recuit simulé, il s'avère moins performant que la méthode taboue et la descente locale, et présente une robustesse moyenne (CV 2,2 %).

— **L'algorithme de colonies de fourmis (ACO)** présente ici des performances contrastées. S'il demeure très stable (CV 0,82 %, le plus faible de toutes les méthodes), il est nettement moins efficace sur le plan du coût, avec des gains limités à +6,27 % (départ glouton) et +1,20 % (départ glouton aléatoire). Cette baisse de performance peut s'expliquer par la difficulté de convergence sur un espace de recherche aussi vaste, la mise à jour des phéromones devenant moins discriminante lorsque le nombre de machines augmente. L'ACO présente toujours l'avantage de sa très grande stabilité, avec des écarts de performance très peu dépendantes du point de départ.

En synthèse, l'augmentation de la taille du problème amplifie les différences de comportement observées sur la petite instance. Les méthodes de **descente locale** et à **plus forte pente** restent très efficaces en temps, mais leur dépendance au point de départ se fait ressentir. La **méthode taboue** s'impose comme la plus équilibrée, offrant un compromis optimal entre qualité, rapidité et robustesse. Le **recuit simulé** maintient des résultats compétitifs au prix d'un temps d'exécution plus élevé, tandis que l'**ACO** conserve une stabilité exemplaire mais perd en efficacité sur des instances de grande taille. La **méthode génétique** permet d'obtenir des résultats similaires à la méthode du recuit-simulé, mais au prix d'un temps d'exécution bien plus faible. Dans l'ensemble, la **méthode de descente à plus forte pente** et la **méthode taboue** apparaissent comme les plus adaptées pour des applications industrielles à grande échelle.

7.5 Graphes de comparaison

Pour rendre ces différences plus visuelles, nous proposons des graphes comparant les gains et le temps d'exécution de chaque méthode, partant des solutions gloutonnes classique et aléatoire, et ce, pour la petite et la grande instance :

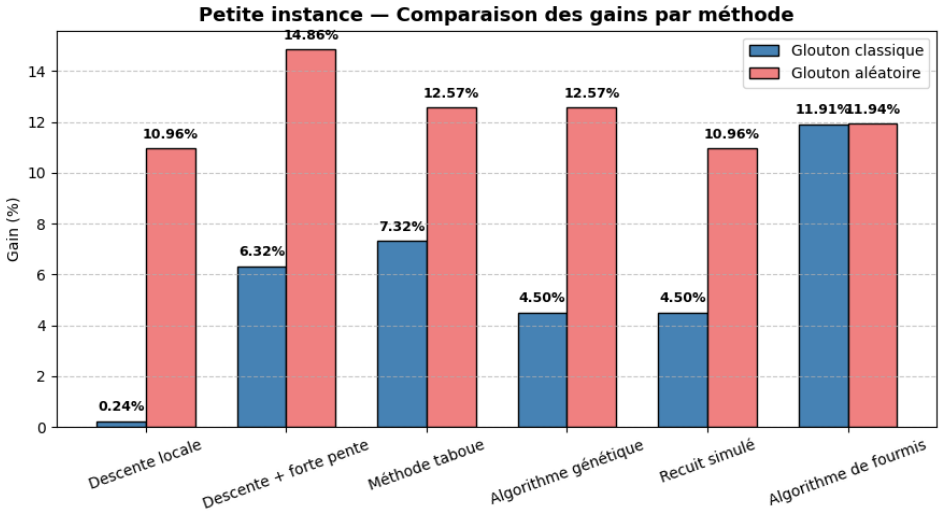


FIGURE 13 – Comparaison Gains : petite instance

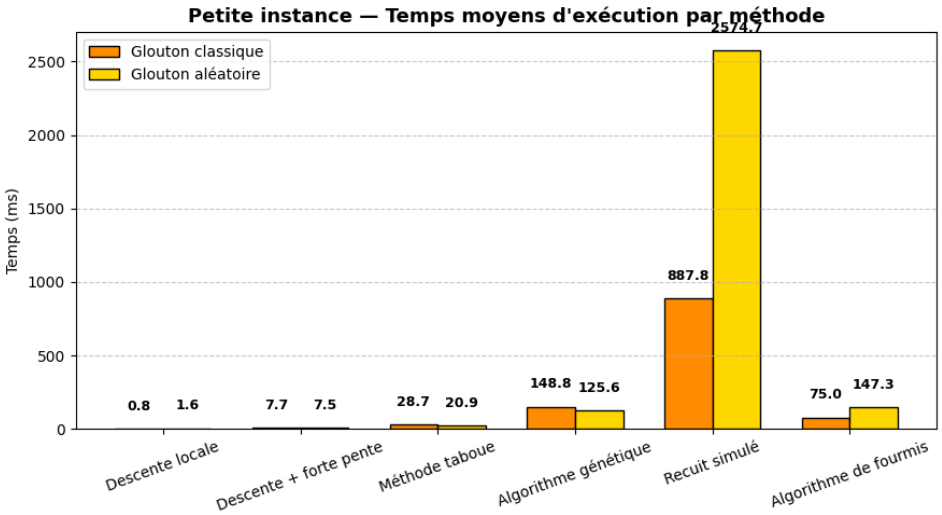


FIGURE 14 – Comparaison Temps : petite instance

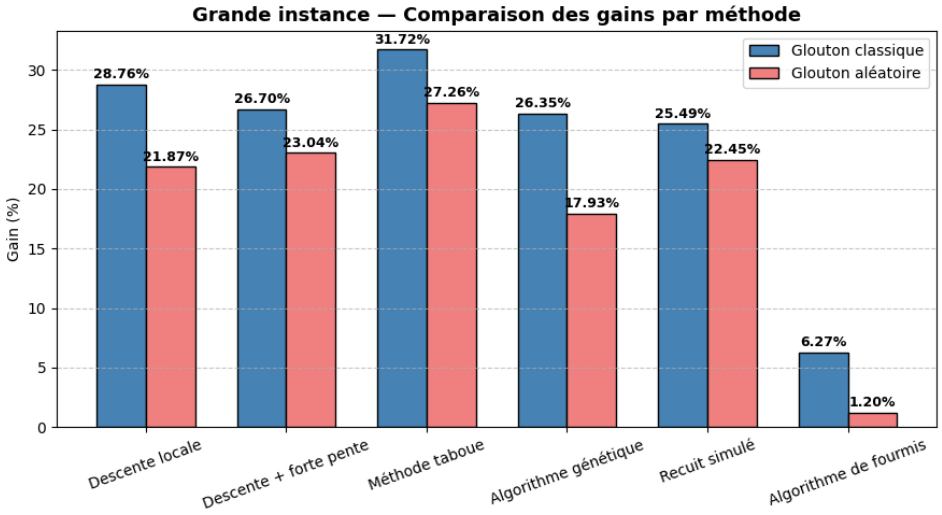


FIGURE 15 – Comparaison Gains : grande instance

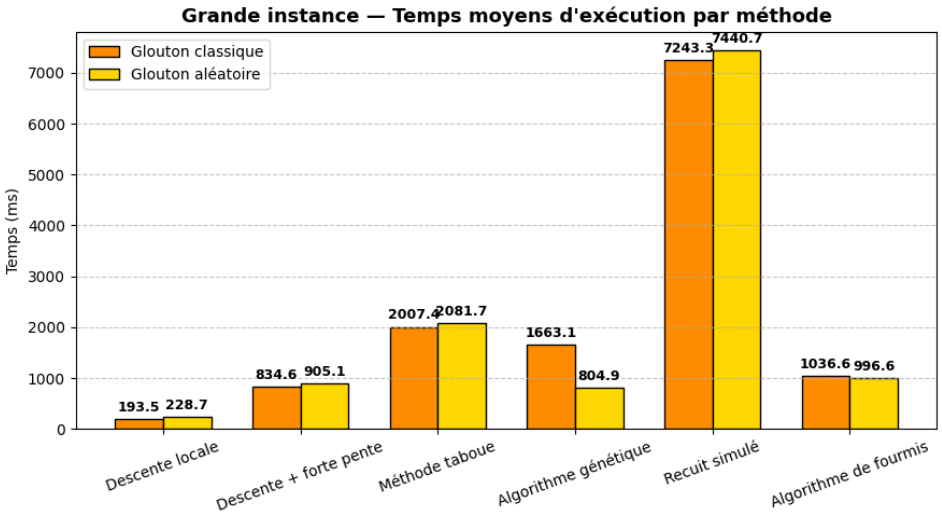


FIGURE 16 – Comparaison Temps : grande instance

Ces figures mettent ici en lumière les conclusions énoncées précedemment, les confirmant alors.

8 Organisation du travail et Coordination

Ce projet a été mené grâce à une collaboration étroite entre tous les membres du groupe. Même si le développement et **l'implémentation technique ont été répartis en tâches spécifiques** (détaillées dans le tableau ci-dessous), **la conception et l'analyse des résultats ont été réalisées ensemble**.

Pour organiser notre travail, nous avons tenu des réunions régulières afin de :

— Définir l'architecture globale du code et vérifier la cohérence des structures de données.

— Étudier les performances des algorithmes et choisir les paramètres à tester.

— Coordonner la rédaction du rapport et synthétiser nos expérimentations.

Le tableau suivant résume la répartition technique des modules développés par chaque membre :

Membre du groupe	**Répartition technique des modules développés**
Mohammed Chali	— Modélisation mathématique du problème — Implémentation de la Solution Gloutonne — Recherche Locale : Insertion à forte pente — Métaheuristique : Algorithme Génétique (AG) — Sensibilité des performances des métaheuristiques aux variations des paramètres — Gestion de la diversité dans l'AG (Doublons)
Clélie Thomas-Suhner	— Modélisation mathématique du problème — Implémentation de la Solution Gloutonne Aléatoire et la stratégie de Multi-initialisation — Recherche Locale : Méthode de descente (2-swap) — Métaheuristique : Méthode Taboue — Analyse des performances sur la Grande Instance
Blandine Piret	— Modélisation mathématique du problème — Recherche Locale : Méthode d'insertion simple — Métaheuristique : Recuit Simulé — Analyse comparative globale des méthodes — Synthèse des résultats
Valéria Ko	— Modélisation mathématique du problème — Métaheuristique : Algorithme de Colonie de Fourmis (ACO) — Réglage des paramètres spécifiques à l'ACO — Synthèse des résultats

TABLE 5 – Répartition des tâches techniques au sein du Groupe

Conformément aux consignes, nous tenons à préciser que **l'unique ajout réalisé après la soutenance finale concerne la gestion de la diversité dans l'Algorithme Génétique (section 6)**. Le reste du travail présenté ici est fidèle à ce qui a été présenté lors de la soutenance.

9 Conclusion

L'objectif de ce travail était de modéliser un problème industriel courant, de découvrir et comprendre le fonctionnement de différentes méthodes gloutonnes, heuristiques et métaheuristiques dans le but de l'optimiser. Ces méthodes ont ensuite été implémentées et appliquées à deux jeux de données conformes à notre problème initial, et ce, dans le but de comparer ces différentes méthodes heuristiques et métaheuristiques. En comparant leur performance sur ces deux instances (la deuxième significativement plus complexe que la première), nous avons pu observer et analyser les comportements de ces méthodes en termes de qualité de solution, de temps de calcul et de robustesse/stabilité.

Sur la petite instance, les méthodes locales (descente simple et à plus forte pente) se sont démarquées, atteignant rapidement des solutions améliorées. Les métaheuristiques, en particulier la méthode taboue et l'algorithme de colonies de fourmis (ACO), ont prouvé leur capacité à dépasser les optimums locaux, la première offrant un excellent compromis entre performance et rapidité. Le recuit simulé, bien que plus lent, était néanmoins efficace en termes de résultat.

Sur la grande instance, la complexité du problème a intensifié les différences entre les méthodes. La descente à plus forte pente et la méthode taboue se sont distinguées par leurs performances globales, la première obtenant le meilleur coût et la seconde la meilleure robustesse. L'ACO, en revanche, a perdu en efficacité à cause de la trop grande taille du problème, minimisant l'impact des phéromones, tandis que le recuit simulé reste performant mais très coûteux en temps de calcul. L'algorithme génétique se caractérise par un bon compromis entre qualité et rapidité, notamment sur la petite instance. Cependant, il perd en efficacité sur les grandes tailles de problème, où les méthodes taboues et à plus forte pente demeurent plus performantes.

En conclusion, ces résultats confirment qu'il n'existe pas un méthode unique se distinguant sur tous les points : chaque méthode possède ses points forts et faiblesse. Le choix d'optimisation dépend alors des priorités de l'entreprise. Cependant, au vu de nos résultats, **nous conseillerions l'industriel d'utiliser la méthode taboue pour optimiser ce problème, cette dernière nous semblant être le meilleur compromis entre gain, temps d'exécution et robustesse**.

Références

[1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983. Accessed via www2.stat.duke.edu (educational reprint).

[2] D. S. Johnson, C. R. Aragon, L. A. McGeoch, and C. Schevon, "Optimization by simulated annealing : An experimental evaluation ; part i, graph partitioning," *Operations Research*, vol. 37, no. 6, pp. 865–892, 1989. Accessible via faculty.washington.edu.

[3] B.-A. Walid, "Computing the initial temperature of simulated annealing," *Computational Optimization and Applications*, vol. 29, no. 3, pp. 369–385, 2004. Available via ACM Digital Library.

[4] B. S. Shweta RaniORCID and R. Goyal, "On the effectiveness of using elitist genetic algorithm in mutation testing," *Symmetry*, vol. 11, no. 9, p. 1145, 2019.

[5] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, United States : The MIT Press, 2004. Disponible en PDF via Carnegie Mellon Qatar : https ://-web2.qatar.cmu.edu/ gdicaro/15382/additional/aco-book.pdf.

[6] M. Mavrovouniotis and S. Yang, "Adapting the pheromone evaporation rate in dynamic routing problems," in *Proceedings of the European Conference on the Applications of Evolutionary Computation (EvoSTOC 2013)*, pp. 606–615, Springer, 2013. Disponible en PDF : https ://mavrovouniotis.github.io/Papers/EVOSTOC13.pdf.

[7] D. Angus and T. Hendtlass, "Dynamic ant colony optimization," in *Proceedings of the 4th International Conference on Ant Colony Optimization and Swarm Intelligence (ANTS 2004)*, pp. 353–360, Springer, 2005. Disponible : https ://researchbank.swinburne.edu.au/items/1b9221e1-62c0-41f1-bc31-b26c4df90b77/.

[8] K. Socha and M. Dorigo, "Ant colony optimization for continuous domains," in *European Journal of Operational Research*, vol. 185, pp. 1155–1173, 2008.

[9] D. Gaertner and K. L. Clark, "On optimal parameters for ant colony optimization algorithms," in *Proceedings of the International Conference on Artificial Intelligence (ICAI 2005)*, pp. 83–89, CSREA Press, 2005.